ObjectSeeker: Certifiably Robust Object Detection against Patch Hiding Attacks via Patch-agnostic Masking

Chong Xiang
Princeton University
cxiang@princeton.edu

Alexander Valtchanov
Princeton University
alexvaltchanov@princeton.edu

Saeed Mahloujifar
Princeton University
sfar@princeton.edu

Prateek Mittal
Princeton University
pmittal@princeton.edu

2023 IEEE Symposium on Security and Privacy (SP) | 978-1-6654-9336-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/SP46215.2023.10179319

***Abstract*—Object detectors, which are widely deployed in security-critical systems such as autonomous vehicles, have been found vulnerable to *patch hiding attacks*. An attacker can use a single physically-realizable adversarial patch to make the object detector miss the detection of victim objects and undermine the functionality of object detection applications. In this paper, we propose ObjectSeeker for certifiably robust object detection against patch hiding attacks. The key insight in ObjectSeeker is *patch-agnostic masking*: we aim to mask out the entire adversarial patch without knowing the shape, size, and location of the patch. This masking operation neutralizes the adversarial effect and allows *any* vanilla object detector to safely detect objects on the masked images. Remarkably, we can evaluate ObjectSeeker's robustness in a certifiable manner: we develop a certification procedure to formally determine if ObjectSeeker can detect certain objects against *any white-box adaptive* attack within the threat model, achieving certifiable robustness. Our experiments demonstrate a significant ($\sim$10%-40% absolute and $\sim$2-6$\times$ relative) improvement in certifiable robustness over the prior work, as well as high clean performance ($\sim$1% drop compared with undefended models).[1]**

1. Introduction

Object detectors, which aim to output a list of bounding boxes detecting every object in a given image, have been shown vulnerable to *patch hiding attacks* [1], [2], [3], [4], [5], [6]. The patch hiding attack aims to make object detectors miss the detection of victim objects (e.g., making an autonomous vehicle miss a pedestrian) using an adversarial *patch* [7]. This attack can be realized in the physical world by attaching the adversarial patch to the real-world scene and thus imposes a notable threat to object detection applications like autonomous driving, augmented reality, and face recognition.

Unfortunately, strong robustness against patch hiding attacks has been hard to obtain. Most existing defenses [8], [9], [10], [11], [12] are based on heuristics and lack formal security guarantees; their claimed robustness could be violated by stronger adaptive attacks. Xiang et al. [13] designed the only certifiably robust defense against patch hiding attacks; nevertheless, the proposed DetectorGuard [13] defense can only provide certifiable robustness for a small number of objects (e.g., $\sim$30% of the objects from the VOC [14] dataset and $\sim$10% of the COCO [15] objects even when the attacker is restricted to place a single 1%-pixel patch *far away* from the victim objects). In this paper, we propose a new approach called ObjectSeeker, which can provide strong certifiable robustness for a significantly larger number of objects (e.g., $\sim$2-6$\times$ improvements over DetectorGuard in our experiments).

ObjectSeeker overview. The core idea of ObjectSeeker is to apply pixel masks to the input image and perform object detection with a vanilla undefended model on the masked images. If the adversarial patch is masked, the attacker has no malicious influence over the model, and any vanilla object detector can safely make predictions. We focus on the setting that the patch hiding attacker can use one adversarial patch whose shape, size, and location are unknown to the defender. We aim to solve two major research questions: (1) How can we generate masks that can remove the patch without knowing the patch shape, size, and location as a priori (i.e., patch-agnostic)? (2) How can we achieve certifiable robustness while maintaining high clean performance? Our ObjectSeeker solution involves two steps of *patch-agnostic masking* and *secure box pruning*; we provide a defense overview in Figure 1.

Step 1: patch-agnostic masking. First, ObjectSeeker applies a set of pixel masks to the input image and performs object detection on different masked images (left of Figure 1). The mask set needs to satisfy two properties. First, some masks from the mask set need to remove the entire adversarial patch (i.e., patch-removing). Second, the mask set generation should not depend on any information on patch shape, size, and location (i.e., patch-agnostic). Once the mask set is generated, this fixed mask set should satisfy the patch-removing property for a patch of different shapes, sizes, and locations.

To attain these two properties, we propose to mask out *image halves*: we divide the image into halves using a fixed set of vertical and horizontal lines, mask out each image half, and perform object detection on the other half. Then, for a patch of any shape, size, and location, we can find lines that do not intersect with the patch to generate image

1. Our source code is available at https://github.com/inspire-group/ObjectSeeker.

DOI 10.1109/SP46215.2023.00078

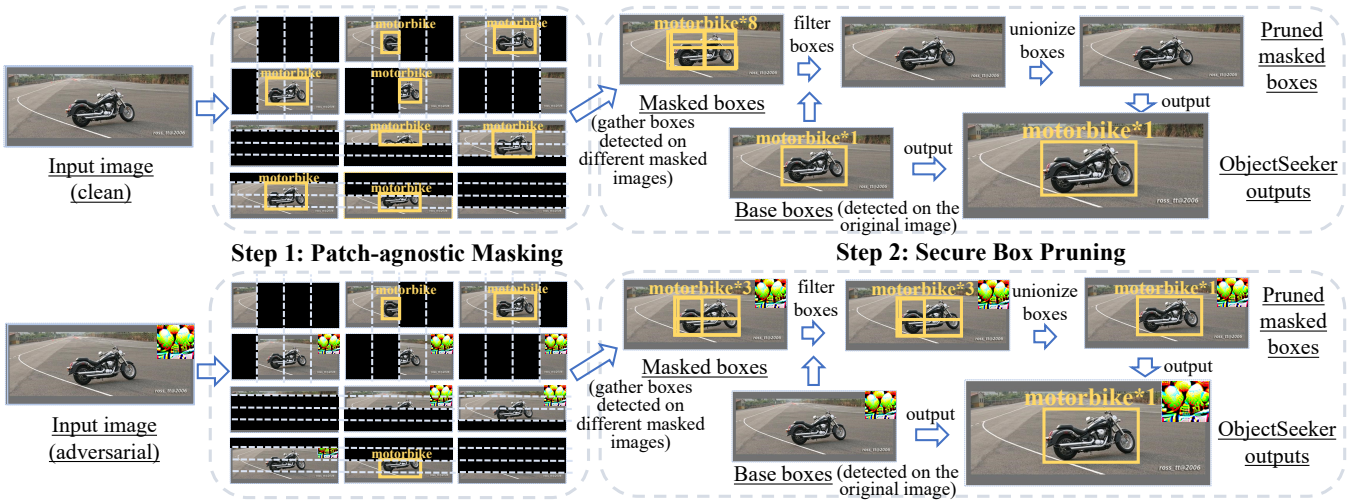

Figure 1. **ObjectSeeker Overview.** *Step 1: patch-agnostic masking* – we mask out image halves divided by a set of vertical and horizontal lines, and perform vanilla object detection on the remaining halves. This masking operation does not need to know the patch shape, size, or location. *Step 2: secure box pruning* – we gather boxes detected on different masked images (*masked boxes*), use a "robust" similarity score to *filter* redundant masked boxes that are duplicates of boxes detected on the original image (*base boxes*), and then *unionize* the remaining masked boxes (if any). Finally, we combine pruned masked boxes and base boxes as the output. In the *clean* setting (top of the figure), the motorbike is detected by one base box and eight masked boxes. All eight masked boxes are pruned, and we finally output the base box. In the *adversarial* setting (bottom of the figure), the motorbike is only detected by three masked boxes (but not base box). No masked box is filtered; we unionize three masked boxes into one and output it as the robust prediction.

halves that can mask out the entire patch. Intuitively, a vanilla object detector is likely to detect the object from the remaining benign pixels, as long as the patch is reasonably sized and does not corrupt the entire victim object. This lays a foundation for robust object detection.

Step 2: secure box pruning. Despite its robustness property, the pixel masking operation can introduce duplicate boxes and downgrade the precision of the model prediction; this is because an object can be detected multiple times on different masked images (e.g., the motorbike in Figure 1 is detected multiple times). The second step of ObjectSeeker aims to remove these redundant boxes for precise object detection outputs while preserving the robustness property achieved by the masking operation. Towards this goal, we use a "robust" box similarity score to identify similar boxes and prune redundant boxes accordingly.

Certifiable robustness evaluation. Remarkably, our ObjectSeeker design allows us to develop a certification procedure for formal robustness evaluation. The procedure will certify if ObjectSeeker can robustly detect a given ground-truth object against any patch attacker within a given threat model (e.g., any attacker placing a 1%-pixel square patch anywhere on the image). Our formal proof in Section 3.3 ensures that the certification results hold for any adaptive attack within the same threat model and thus allows us to evaluate robustness with absolute certainty.

State-of-the-art certified robustness across datasets. We implement ObjectSeeker with two vanilla object detectors: YOLOR [16] and Mask R-CNN [17] with a Swin Transformer backbone [18], and evaluate it on three object detection datasets: VOC [14], COCO [15], and KITTI [19]. We demonstrate that ObjectSeeker achieves significant (∼10%-40% absolute and ∼2-6× relative) improvements in certified robustness over DetectorGuard [13]. In the meantime, ObjectSeeker also has high clean performance similar to that of vanilla models (∼1% clean performance drops). Our contributions can be summarized as follows.

- We propose ObjectSeeker as a certifiably robust defense framework for any vanilla object detector via a patch-agnostic masking strategy.
- We develop a certification procedure to determine if ObjectSeeker is provably robust, for a given object, against any attack within a given threat model.
- We evaluate ObjectSeeker on object detection benchmark datasets and demonstrate significant robustness improvements over DetectorGuard [13].

2. Background and Problem Formulation

In this section, we introduce the task of object detection, attack formulation, and defense formulation.

2.1. Object Detection

The task of object detection is to predict a list of bounding boxes that locate and classify each object in a given image. We use $\mathbf{x} \in \mathcal{X} \subset [0,1]^{W \times H \times C}$ to denote the input image with width W, height H, and C color channels. We use $\mathbf{b} = (x_{\min}, y_{\min}, x_{\max}, y_{\max}, \ell, c)$ to represent a bounding box, where four coordinates $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ illustrate the box, ℓ denotes the class label, and $c \in [0,1]$ denotes the prediction confidence for this box detection. An object detector takes an image $\mathbf{x}$ as the input and outputs a list of boxes $\mathbf{b}_0, \mathbf{b}_1, \cdots, \mathbf{b}_n$. We further use an unordered set $\mathcal{B} = \{\mathbf{b}_0, \mathbf{b}_1, \cdots, \mathbf{b}_n\}$ to denote the detection output and let $\mathcal{O}$ denote the space of all possible $\mathcal{B}$ (detection

outputs). We can then formally represent an object detector as $\mathbb{F}(\mathbf{x}, \gamma) : \mathcal{X} \times [0,1] \rightarrow \mathcal{O}$, where the detector only outputs boxes with confidence higher than γ. We do not make any assumption on the object detector $\mathbb{F}$, it can be any off-the-shelf model such as YOLO [20], [21], [16], Faster R-CNN [22], and Mask R-CNN [17].

Box operation. In some cases, we abuse the notation $\mathbf{b}$ by considering $\mathbf{b}$ as a set of image pixels within the box. Then, we can define several important box operations: $|\mathbf{b}|$ denotes the *area* of the box $\mathbf{b}$; $\mathbf{b}_0 \cap \mathbf{b}_1$ and $\mathbf{b}_0 \cup \mathbf{b}_1$ denote the *intersection* and *union* of two boxes, respectively; $\mathbf{b}_0 \backslash \mathbf{b}_1$ denotes the image region that belongs to $\mathbf{b}_0$ but not $\mathbf{b}_1$. We only consider operations between boxes when they share the same class labels. When $\mathbf{b}_0, \mathbf{b}_1$ have different class labels, $\mathbf{b}_0 \cap \mathbf{b}_1 = \varnothing$; $\mathbf{b}_0 \cup \mathbf{b}_1$ and $\mathbf{b}_0 \setminus \mathbf{b}_1$ are undefined.

Performance evaluation. To evaluate a conventional object detector, we consider a detected box is correct if (1) the box class label matches the ground-truth box label, and (2) Intersection over Union (IoU) between the detected box and the ground-truth box, defined as $|\mathbf{b}_0 \cap \mathbf{b}_1| / |\mathbf{b}_0 \cup \mathbf{b}_1|$, exceeds a certain threshold [14], [15]. We count a correct detected box as a true-positive (TP). On the other hand, any detected box that is not a TP is a false-positive (FP); any ground-truth box that is not correctly detected is a false-negative (FN). Furthermore, we define *precision* as TP/(TP+FP) and *recall* as TP/(TP+FN). We aim to build object detectors that have both high precision and recall.

2.2. Attack Formulation

We focus on defending against patch hiding attacks.

Objective: hiding attacks. The hiding attack aims to make the object detector miss the detection of the victim object, which increases FN errors and downgrades the detection recall. This attack can cause serious consequences in real-world scenarios such as an autonomous car failing to detect and consequently hitting a pedestrian.

Means: patch attacks. The patch attack aims to generate adversarial pixels within a local region (forming a patch) to cause mispredictions of the machine learning model. This attack can be realized in the physical world by printing and attaching the patch to the underlying scene and thus imposes an urgent threat to real-world machine learning applications. Formally, we denote the patch region with a binary tensor $\mathbf{r} \in \{0,1\}^{W \times H}$ of the same shape as the $W \times H$ input image $\mathbf{x}$. We set the elements within the patch region to zeros, and the rest to ones. We further use $\mathcal{R}$ to denote a set of patch regions $\mathbf{r}$ that an attacker can use. Then, the constraint set of the patch attack can be represented as $\mathcal{A}_{\mathcal{R}}(\mathbf{x}) = \{\mathbf{x}' = \mathbf{x} \odot \mathbf{r} + \mathbf{x}'' \odot (\mathbf{1} - \mathbf{r}) | \mathbf{x} \in \mathcal{X}, \mathbf{r} \in \mathcal{R}\}$. $\odot$ refers to the element-wise product operator. $\mathbf{x}'' \in [0,1]^{W \times H \times C}$ is the patch content *arbitrarily* controlled by the attacker.

Threat model. We note that the patch region set $\mathcal{R}$ is determined by the number of patches, patch shape, patch size, and patch location. In this paper, we primarily focus on the open research question of *one* adversarial patch; nevertheless, we will quantitatively discuss defenses against multiple patches in Section 5.

Furthermore, we allow the patch to have different *shapes* such as square, rectangle, and circle. The patch *size* can take any *reasonable* value as long as the patch does not occlude the entire object; otherwise, the defense problem is meaningless since no one (not even a human) can detect a completely invisible object.

Finally, we divide all possible patch *locations* into three groups: far-patch, close-patch, and over-patch when the patch is far away from, close to, and over the object. The attacker can pick *any* location within a certain location set. These different location threat models represent different attackers in the physical world: for example, whether being able to place the patch over the victim object or not clearly exhibits different attacker capabilities.

In our evaluation, we will consider different patch region sets $\mathcal{R}$ such as a set of single 2%-pixel square patches at all possible locations over the victim object or single 1%-pixel rectangle patches anywhere far away from the object.

2.3. Defense Formulation

In this section, we formulate the defense problem.

Defender knowledge (patch-agnostic). As discussed in Section 2.2, we consider patch hiding attacks that use one adversarial patch. The defender only knows that there might be one patch on the input image, but does not know anything about the patch shape, size, and location (i.e., patch-agnostic). This requires the defender to build a defense with a *fixed* set of parameters to deal with *different* patch attackers that use different patch region sets $\mathcal{R}$.

Note: The assumption of a *reasonable patch size* discussed in the last subsection will not give an additional advantage to our defense or violate the patch-agnostic property: it only ensures the object visibility that is required by any object detector (including humans).

Robustness definition. Facing a patch hiding attack, our defense aims to robustly predict bounding boxes that *cover at least part of each object and have correct class labels*. To formally discuss the robustness objective, we introduce the following concept of Intersection over Area (IoA)

Definition 1 (Intersection over Area (IoA))**.** *The IoA between two boxes* $\mathbf{b}_0$ *and* $\mathbf{b}_1$ *is the ratio of the intersecting area of two regions to the area of the first box* $\mathbf{b}_0$*. Formally, we have:* $\text{IoA}(\mathbf{b}_0, \mathbf{b}_1) = |\mathbf{b}_0 \cap \mathbf{b}_1| / |\mathbf{b}_0|$.

We note that when two boxes have different class labels, we consider the intersection $\mathbf{b}_0 \cap \mathbf{b}_1$ empty and the IoA is 0.

With the concept of IoA, we can then reiterate the robustness definition as: given a ground-truth box $\mathbf{b}_{\text{gt}}$, we aim to detect a box $\mathbf{b}'$ such that $\text{IoA}(\mathbf{b}_{\text{gt}}, \mathbf{b}') > T$, where $T \in [0,1]$ determines the strength of robustness. We term this as *IoA robustness*. We choose this robustness objective because it aligns with our goal of mitigating hiding attacks; this objective is also similar to that of DetectorGuard [13] and enables a fair comparison. In Appendix B and C, we provide discussions on other possible robustness definitions.

Certifiable robustness evaluation. More importantly, we target *certifiable robustness* – evaluating robustness in a

TABLE 1. SUMMARY OF IMPORTANT NOTATION

Notation	Description	Notation	Description
$\mathbf{x} \in \mathcal{X}$	input image	$\mathbf{b} \in \mathcal{B} \in \mathcal{O}$	bounding box
$\mathbf{m} \in \mathcal{M}$	pixel mask	$\mathbf{r} \in \mathcal{R}$	patch region
$\mathbb{F} : \mathcal{X} \times [0,1] \to \mathcal{B}$	undefended model	$\gamma \in [0,1]$	confidence thres.
$\mathbb{S} : \mathcal{B} \times \mathcal{B} \to \mathbb{R}$	similarity score	$\tau \in [0,1]$	filtering thres.
$k \in \mathbb{Z}^+$	# lines	$T \in [0,1]$	certification thres.

certifiable manner. We will develop a certification procedure to determine if ObjectSeeker can robustly detect a given ground-truth object against a given threat model. We will formally prove that the certification procedure accounts for all possible attackers within the given threat model $\mathcal{A}_{\mathcal{R}}$, including an adaptive attacker with perfect knowledge of our defense setup. In our evaluation, we apply the certification procedure to datasets with annotated ground-truth bounding boxes (objects) and use *certified recall*, the fraction of certified objects among all ground-truth objects, as our robustness metric.

Remark: patch-agnostic inference vs. certification. We note that our patch-agnostic property is only for inference but not certification. The inference procedure (Algorithm 1) is the defense that will be *deployed in practice*. Patch-agnostic inference allows us to use the same inference setting to achieve non-trivial robustness against different patch shapes/sizes/locations (different patch region sets $\mathcal{R}$). In contrast, the certification procedure (Algorithm 2) is for *evaluating robustness* against *one* specific patch threat model $\mathcal{A}_{\mathcal{R}}$, and thus requires patch information. In fact, "patch-agnostic certification" is impossible: we cannot certify robustness against an unknown/unbounded attack capability.

3. ObjectSeeker Design

Overview. In this section, we introduce the design of ObjectSeeker, a defense framework for building certifiably robust object detectors against patch hiding attacks. ObjectSeeker operates in a two-step manner (recall the defense overview in Figure 1). First, ObjectSeeker applies a set of pixel masks to the input images and performs vanilla object detection on masked images (Section 3.1). Our masking strategy ensures that some of the masks remove the entire adversarial patch so that a vanilla object detector can detect victim objects from the unmasked regions of these images. This masking operation lays a foundation for robustness; however, it may introduce redundant boxes when the same object is repeatedly detected on multiple masked images. The second step of ObjectSeeker aims to remove these redundant boxes via secure box pruning (Section 3.2); this will improve the detection precision while preserving the robustness property achieved by the masking operation. Notably, with a careful design of pixel masking and box pruning, we can develop a certification procedure to determine if ObjectSeeker has provable robustness guarantees for certain objects against any adaptive attacker within a given threat model (Section 3.3), achieving certifiable robustness.

We provide the pseudocode of ObjectSeeker in Algorithm 1 and a summary of important notation in Table 1.

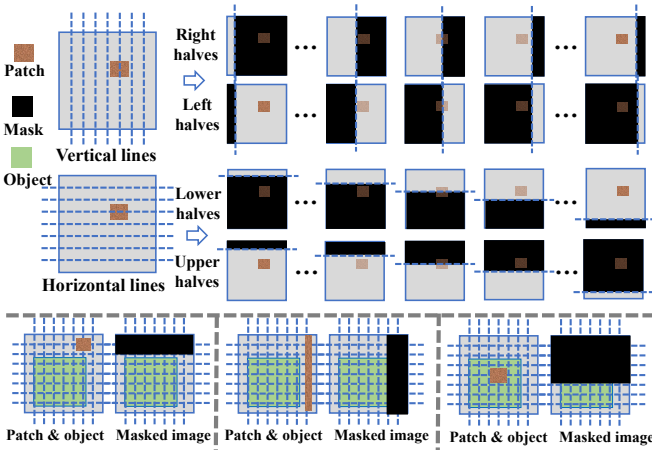

Figure 2. **Visualization for patch-agnostic masking:** the algorithm masks out image halves divided by vertical/horizontal lines (top of the figure); masks from a fixed mask set can remove patches of different shapes, sizes, and locations (bottom of the figure).

We will discuss the details of each defense module next.

3.1. Patch-agnostic Masking

Intuition. The objective of our pixel-masking defense is to find masks to mask out the entire patch (but not the entire victim objects) from the input image. Typically, vanilla object detectors can safely detect objects from the masked image once all adversarial pixels are removed.

Challenge. If we have information on patch shapes, sizes, and locations, it is easy to find masks to remove the patch. For example, if we know an attacker will use a 32×32 *patch*, we can enumerate all possible 32×32 *masks* at different locations, and one of the masks must remove the entire 32×32 *patch*. However, when we do not have patch information, this enumeration no longer works due to an infinite number of patch regions $\mathbf{r}$. In ObjectSeeker, we propose an alternative masking strategy that does not depend on the patch information (i.e., patch-agnostic).

Masking strategy. The robustness requirement of our masking algorithm is that: for each object, we have at least one mask that removes the entire patch while preserving most of the object pixels (so that we can safely detect each object). Towards this goal, we propose to mask out *"image halves"* determined by a set of horizontal and vertical lines; we provide visualization for this strategy in Figure 2. First, we select a set of evenly spaced horizontal and vertical lines across the image (top left of Figure 2). Second, observing that each horizontal/vertical line splits the image into two halves, we mask out one half and perform vanilla object detection on the other half (top right of Figure 2).

The patch-agnostic property. Clearly, our mask set generation strategy only requires the number of lines as its input, and does not rely on any information of patch shape/size/location (i.e., patch-agnostic). For a patch of any shape, size, and location, vertical/horizontal lines that *do not intersect with the patch* can generate masks that remove the entire patch. At the bottom of Figure 2, we visualize

Algorithm 1 ObjectSeeker inference algorithm

Input: Image $\mathbf{x}$, image size (W, H), vanilla object detector $\mathbb{F}$, number of lines k (for masking), masked box confidence threshold γ_{m}, base box confidence threshold γ_{b}, similarity score function $\mathbb{S}$, box filtering threshold τ

Output: Robust detection results $\mathcal{B}_{\text{robust}}$

1: **procedure** OBJECTSEEKER($\mathbf{x}, \mathbb{F}, k, W, H, \gamma_{\text{m}}, \gamma_{\text{b}}, \mathbb{S}, \tau$)
2: $\quad\mathcal{M} \leftarrow$ MASKSET(k, W, H)
3: $\quad\mathcal{B}_{\text{mask}} \leftarrow \varnothing$
4: $\quad$**for** $\mathbf{m} \in \mathcal{M}$ **do**
5: $\quad\quad\mathcal{B}_{\mathbf{m}} \leftarrow \mathbb{F}(\mathbf{x} \odot \mathbf{m}, \gamma_{\text{m}})$
6: $\quad\quad\mathcal{B}_{\text{mask}} \leftarrow \mathcal{B}_{\text{mask}} \cup \mathcal{B}_{\mathbf{m}}$
7: $\quad$**end for**
8: $\quad\mathcal{B}_{\text{base}} \leftarrow \mathbb{F}(\mathbf{x}, \gamma_{\text{b}})$
9: $\quad\mathcal{B}_{\text{robust}} \leftarrow$ BOXPRUNE$(\mathcal{B}_{\text{mask}}, \mathcal{B}_{\text{base}}, \mathbb{S}, \tau)$
10: $\quad$**return** $\mathcal{B}_{\text{robust}}$
11: **end procedure**

12: **procedure** MASKSET(k, W, H)
13: $\quad\mathcal{X} = \{\lceil \frac{W}{k+1} \rceil \cdot t | t \in \mathbb{Z}_k^+\};\ \mathcal{Y} = \{\lceil \frac{H}{k+1} \rceil \cdot t | t \in \mathbb{Z}_k^+\}$
14: $\quad\mathcal{M}_{\mathcal{X}} \leftarrow \{\mathbf{m} | \mathbf{m}[i,j] = 0, i \in [0, x); \mathbf{m}[i,j] = 1, i \in [x, W), x \in \mathcal{X}\}$
15: $\quad\mathcal{M}_{\mathcal{Y}} \leftarrow \{\mathbf{m} | \mathbf{m}[i,j] = 0, j \in [0, y); \mathbf{m}[i,j] = 1, j \in [y, H), y \in \mathcal{Y}\}$
16: $\quad\bar{\mathcal{M}}_{\mathcal{X}} \leftarrow \{\mathbf{1} - \mathbf{m} | \mathbf{m} \in \mathcal{M}_{\mathcal{X}}\};\ \bar{\mathcal{M}}_{\mathcal{Y}} = \{\mathbf{1} - \mathbf{m} | \mathbf{m} \in \mathcal{M}_{\mathcal{Y}}\}$
17: $\quad$**return** $\mathcal{M}_{\mathcal{X}} \cup \mathcal{M}_{\mathcal{Y}} \cup \bar{\mathcal{M}}_{\mathcal{X}} \cup \bar{\mathcal{M}}_{\mathcal{Y}}$
18: **end procedure**

19: **procedure** BOXPRUNE$(\mathcal{B}_{\text{mask}}, \mathcal{B}_{\text{base}}, \mathbb{S}, \tau)$
20: $\quad\mathcal{B}_{\text{mask}}^{\text{filtered}} \leftarrow \{\mathbf{b}_{\text{m}} \in \mathcal{B}_{\text{mask}} | \nexists\ \mathbf{b}_{\text{b}} \in \mathcal{B}_{\text{base}} : \mathbb{S}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau\}$
21: $\quad\mathcal{B}_{\text{mask}}^{\text{pruned}} \leftarrow \{\text{REP}(\hat{\mathcal{B}}, \mathbb{S})) \mid \hat{\mathcal{B}} \in \text{CLUSTER}(\mathcal{B}_{\text{mask}}^{\text{filtered}}, \mathbb{S})\}$
22: $\quad\mathcal{B}_{\text{robust}} \leftarrow \mathcal{B}_{\text{base}} \cup \mathcal{B}_{\text{mask}}^{\text{pruned}}$
23: $\quad$**return** $\mathcal{B}_{\text{robust}}$
24: **end procedure**

three patches of different shapes, sizes, and locations and the corresponding masks that can remove the patch. We can see that masks from our *fixed* mask set can remove *different* patches while preserving a large portion of object pixels. This lays a foundation for robust object detection. Finally, we highlight that the patch-agnostic property is a significant improvement from all existing masking-based certifiably robust defenses (for image classification) [23], [24], [25], [26], which require patch information (e.g., sizes, shapes) for their inference procedure to achieve non-trivial certifiable robustness.

Masking pseudocode. Now, we discuss the implementation details of patch-agnostic masking. We present the pseudocode in Line 2-7 of Algorithm 1. The first step of pixel masking is to generate a mask set (Line 2). Formally, we represent each mask as a binary tensor $\mathbf{m} \in \mathcal{M} \subset \{0,1\}^{W \times H}$ that has the same shape as the $W \times H$ images. We set the elements within the mask to 0, and others to 1.

We use the sub-procedure MASKSET$(\cdot)$ for the mask set generation. It takes the image size $W \times H$ and the number of horizontal/vertical lines k as inputs and outputs a mask set $\mathcal{M}$. In this sub-procedure, we first generate two sets of coordinates $\mathcal{X}, \mathcal{Y}$ that contain k evenly spaced coordinates along two image axes (Line 13). Next, we generate a mask set $\mathcal{M}_{\mathcal{X}}$, whose elements mask the left halves of the image (Line 14). Similarly, we generate $\mathcal{M}_{\mathcal{Y}}$ that masks the lower halves, $\bar{\mathcal{M}}_{\mathcal{X}}$ that masks the right halves, and $\bar{\mathcal{M}}_{\mathcal{Y}}$ that masks the upper halves (Line 15-16). The final mask set $\mathcal{M}$ is the union of these four sets (Line 17).

With the generated mask set $\mathcal{M}$, we iterate over each mask $\mathbf{m} \in \mathcal{M}$ (Line 4), perform object detection with an undefended model $\mathbb{F}(\cdot, \gamma_{\text{m}})$ on the masked image $\mathbf{x} \odot \mathbf{m}$ (Line 5), and then gather detected boxes into the box set $\mathcal{B}_{\text{mask}}$ (Line 6). After that, ObjectSeeker will perform secure box pruning on these detected boxes, which will be discussed in the next subsection.

3.2. Secure Box Pruning

Intuition. Our masking operation allows us to safely see benign pixels/objects from some of the masked images that contain no adversarial pixels. With this robustness property, a naive defense strategy is to take all boxes detected on masked images as the final output. Since the attacker has no influence over the detection results when the patch is completely masked, we will detect most ground-truth objects and achieve a high robust *recall* against the hiding attack.

Challenge. However, this approach is not ideal: we will have many duplicate boxes for one object since an object might be detected multiple times in different masked images (see "masked boxes" in Figure 1 for visual examples); redundant duplicate boxes will be considered as false-positive errors and hurt the detection *precision*. Therefore, we need to further identify and prune these redundant boxes in ObjectSeeker. We note that box pruning is a non-trivial task since duplicate boxes can look very different (e.g., in Figure 1, some boxes detect the left part of the motorbike while some detect the right part). Moreover, the box pruning should be done in a secure manner since an adaptive attacker might introduce malicious boxes in some masked images (where the patch is not completely removed) to interfere with the box pruning. In summary, we need to perform secure box pruning to improve prediction *precision* while preserving the robust *recall* against the patch hiding attack.

Box pruning algorithm. The high-level idea of our box pruning is to use a score function $\mathbb{S} : \mathcal{B} \times \mathcal{B} \rightarrow \mathbb{R}$ to robustly measure the similarity between detected boxes so that we can identify and prune redundant boxes. We note that different similarity scores can give different robustness guarantees. Here, we focus on using IoA as the similarity score for achieving IoA robustness (recall Section 2.3); we will also discuss an alternative choice in Appendix B.

We visualize the pruning algorithm in the right of Figure 1 and present the pseudocode in Line 8-9 and Line 20-22 of Algorithm 1. To start with, we perform one vanilla model prediction on the original unmasked image $\mathbf{x}$ to obtain the (potentially vulnerable) base detection results

$\mathcal{B}_{\text{base}} = \mathbb{F}(\mathbf{x}, \gamma_{\text{b}})$ (Line 8). We term boxes detected on the original unmasked image *base boxes* and boxes detected on masked images as *masked boxes*. Then, we call the sub-procedure $\text{BOXPRUNE}(\mathcal{B}_{\text{mask}}, \mathcal{B}_{\text{base}}, \mathbb{S}, \tau)$ for box pruning, which first *filters* out redundant masked boxes that are duplicates of base boxes and then *unionizes* unfiltered masked boxes for the final prediction output.

Box filtering. The first pruning operation aims to filter out redundant masked boxes that are highly similar to the base boxes (measured by $\mathbb{S}$). Specifically, we calculate the pairwise similarity scores between masked boxes and base boxes and remove masked boxes $\mathbf{b}_{\text{m}}$ whose similarity with a particular base box $\mathbf{b}_{\text{b}}$ exceeds a filtering threshold τ, i.e., $\mathbb{S}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau$. This filtering operation is illustrated in Line 20 of Algorithm 1. In the top right of Figure 1, we can see that duplicate boxes are effectively removed when base boxes also detect the object.

Box unionizing. Intuitively, if base box predictions are accurate, most masked boxes will be removed during the box filtering operation, and we will directly output high-quality base boxes (top right of Figure 1). However, when a patch hiding attack happens, the attacker makes base box predictions disappear, and thus no masked boxes will be filtered (bottom right of Figure 1). As a result, we need to further prune/unionize the remaining redundant masked boxes. The high-level idea of box unionizing is to use similarity score $\mathbb{S}$ to cluster similar boxes via $\text{CLUSTER}(\cdot)$ and output one box representative for each cluster via $\text{REP}(\cdot)$. The collection of these box representatives is the final *pruned masked boxes*. This process occurs in Line 21 of Algorithm 1 and is illustrated in the bottom right of Figure 1.

ObjectSeeker output. After the box filtering and unionizing operations, we combine the pruned masked boxes $\mathcal{B}_{\text{mask}}^{\text{pruned}}$ with base boxes $\mathcal{B}_{\text{base}}$ as the final output of ObjectSeeker (Line 23 and Line 10).

Instantiation with IoA. In our implementation when we take IoA as $\mathbb{S}$, we instantiate $\text{CLUSTER}(\cdot)$ using a distance-based clustering algorithm DBSCAN [27]; the "distance" between boxes $\mathbf{b}_0, \mathbf{b}_1$ is calculated as $1 - \max(\text{IOA}(\mathbf{b}_0, \mathbf{b}_1), \text{IOA}(\mathbf{b}_1, \mathbf{b}_0))$. For each box cluster, we generate the box representative as $\text{REP}(\hat{\mathcal{B}}) = \bigcup_{\mathbf{b} \in \hat{\mathcal{B}}} \mathbf{b}$, i.e., taking the mathematical union of all boxes as the representative of the cluster. In the rest of this paper, we will refer to this instance of box pruning as $\textbf{IOA-BOXPRUNE}(\mathcal{B}_{\textbf{mask}}, \mathcal{B}_{\textbf{base}}, \tau)$ and call the corresponding ObjectSeeker instance $\textbf{IOA-OBJECTSEEKER}(\mathbf{x})$.

3.3. Robustness Certification

So far, we have discussed how to use patch-agnostic masking to neutralize the adversarial patch and how to securely perform box pruning to remove duplicate boxes. In this subsection, we develop the robustness certification procedure (Algorithm 2) for provable robustness evaluation. Given a victim object and a specific threat model (e.g., a specific patch size, shape, and location set), the certification procedure aims to determine if ObjectSeeker can robustly detect the object against all possible attackers within a given threat model. Here, we will focus on the case where we implement $\mathbb{S}$ with IoA for certifiable IoA robustness. We note that the high-level idea of certification is similar for different $\mathbb{S}$; we will discuss a different $\mathbb{S}$ and its certification in Appendix B.

Algorithm 2 Certification algorithm for IoA robustness

Input: Image $\mathbf{x}$, the object (bounding box) $\mathbf{b}_{\text{gt}}$ to be certified, valid patch region set $\mathcal{R}$, defense setup $(\mathbb{F}, \mathcal{M}, \gamma_{\text{m}}, \tau)$, certification threshold T.

Output: Whether ObjectSeeker has certifiable IoA robustness for $\mathbf{b}_{\text{gt}}$ against $\mathcal{A}_{\mathcal{R}}$

1: **procedure** $\text{CERTIFY}(\mathbf{x}, \mathbf{b}, \mathcal{R}, \mathbb{F}, \mathcal{M}, \gamma_{\text{m}}, \tau)$
2: **for** every $\mathbf{r} \in \mathcal{R}$ **do**
3: $f_{\mathbf{r}} \leftarrow$ `False`
4: **for** $\mathbf{m} \in \{\mathbf{m} \in \mathcal{M} | \mathbf{m}[i,j] \le \mathbf{r}[i,j], \forall (i,j)\}$ **do**
5: $\mathcal{B}_{\mathbf{m}} \leftarrow \mathbb{F}(\mathbf{x} \odot \mathbf{m}, \gamma_{\text{m}})$
6: **if** $\exists \mathbf{b}_{\mathbf{m}} \in \mathcal{B}_{\mathbf{m}}$ s.t. $\frac{|\mathbf{b}_{\text{m}}| \cdot \tau - |\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}}|}{|\mathbf{b}_{\text{gt}}|} > T$ **then**
7: $f_{\mathbf{r}} \leftarrow$ `True`; **break**
8: **end if**
9: **end for**
10: **if** $f_{\mathbf{r}} =$ `False` **then**
11: **return** `False`
12: **end if**
13: **end for**
14: **return** `True`
15: **end procedure**

We first formally reiterate our IoA robustness objective.

Definition 2 (Certifiable IoA robustness)**.** *We consider ObjectSeeker is certifiably IoA-robust for a ground-truth object $\mathbf{b}_{gt}$ on an image $\mathbf{x}$ against a patch attacker $\mathcal{A}_{\mathcal{R}}$ (discussed in Section 2.2), if we can always predict a box $\mathbf{b}'$ satisfying $\text{IOA}(\mathbf{b}_{gt}, \mathbf{b}') > T$, where $T \in [0,1]$ is a certification threshold. Formally, we have: $\forall\ \mathbf{x}' \in \mathcal{A}_{\mathcal{R}}(\mathbf{x}), \exists\ \mathbf{b}' \in \mathcal{B}_{robust} = \text{IOA-OBJECTSEEKER}(\mathbf{x}')$ s.t. $\text{IOA}(\mathbf{b}_{gt}, \mathbf{b}') > T$.*

Next, we discuss our certification intuition, present our certification procedure in Algorithm 2, and prove its soundness in Theorem 1.

Certification intuition. In our ObjectSeeker framework, we aim to detect victim objects on *masked images without adversarial pixels* and combine pruned masked boxes and base boxes as the final output $\mathcal{B}_{\text{robust}}$. Intuitively, if we have a "good" masked box in $\mathcal{B}_{\text{mask}}$, this good masked box is likely to "survive" the box pruning procedure and become one good box in $\mathcal{B}_{\text{robust}}$. Our certification aims to search for "good" boxes on masked images without adversarial pixels.

Certification algorithm. We provide the certification pseudocode in Algorithm 2. It determines if ObjectSeeker has certifiable IoA robustness for the given ground-truth object $\mathbf{b}_{\text{gt}}$ against the given attack threat model $\mathcal{A}_{\mathcal{R}}$.

Overall, the certification algorithm will iterate every valid patch region $\mathbf{r} \in \mathcal{R}$ (e.g., every valid patch location) to determine if ObjectSeeker can certify the robustness for the ground-truth object $\mathbf{b}_{\text{gt}}$ against every $\mathbf{r}$ (Line 3-9).

For each patch region $\mathbf{r}$ that represents a specific patch

shape, size, and location, we initialize the robustness flag $f_{\mathbf{r}}$ to `False` (Line 3). Next, we will examine every mask $\mathbf{m} \in \mathcal{M}$ that can remove the entire patch (i.e., $\mathbf{m}[i,j] \leq \mathbf{r}[i,j], \forall (i,j)$). For each mask $\mathbf{m}$, we perform object detection on the masked image $\mathbf{x} \odot \mathbf{m}$ to get masked boxes $\mathcal{B}_{\mathbf{m}}$ (Line 5). Then, we will search for any "good" box in $\mathcal{B}_{\mathbf{m}}$. If a masked box $\mathbf{b_m} \in \mathcal{B}_{\mathbf{m}}$ satisfies $\frac{|\mathbf{b}_{\text{m}}| \cdot \tau - |\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}}|}{|\mathbf{b}_{\text{gt}}|} > T$, we consider this box "certifiably good" for robustly detecting object $\mathbf{b}_{\text{gt}}$ against patch region $\mathbf{r}$ (will be proved in Theorem 1). On the other hand, if we try all possible masks that can remove the patch, and no masked box satisfies this condition, the object $\mathbf{b}_{\text{gt}}$ might be vulnerable to this patch region $\mathbf{r}$ and the procedure returns `False` (Line 11).

Finally, if the certification procedure enumerates *every* valid patch $\mathbf{r} \in \mathcal{R}$ and does not return `False`, it implies that ObjectSeeker has certified robustness for the object $\mathbf{b}_{\text{gt}}$ against all possible $\mathbf{r}$ within the threat model $\mathcal{A}_{\mathcal{R}}$. The algorithm returns `True` (Line 14).

Soundness of Algorithm 2. We present Theorem 2 below to prove the soundness of our certification.

Theorem 1. *Given a ground-truth object box $\mathbf{b}_{gt}$ in the input image $\mathbf{x}$, defense setup $(\mathbb{F}, \mathcal{M}, \gamma_m, \tau)$, certification threshold T, and a set of valid patch regions $\mathcal{R}$, if Algorithm 2 returns* `True`*,* IOA-OBJECTSEEKER *has certifiable IoA robustness for the object $\mathbf{b}_{gt}$ against any attacker in $\mathcal{A}_{\mathcal{R}}$.*

Proof. Recall our certification intuition: if we can detect a "good" masked box $\mathbf{b}_{\text{m}} \in \mathcal{B}_{\text{mask}}$ on images without adversarial pixels, this box is likely to "survive" box pruning and become a good box in $\mathcal{B}_{\text{robust}}$. In this proof, we will first define the property of a "good" box as *pruning-safe* masked box, and then discuss how to find pruning-safe boxes.

Definition 3 (pruning-safe masked box)**.** *Let $\mathbf{b}_m$ be a masked box that is part of the box pruning input $\mathcal{B}_{mask}$. We call $\mathbf{b}_m$ a pruning-safe masked box for the ground-truth object $\mathbf{b}_{gt}$ and pruning procedure* IOA-BOXPRUNE$(\cdot,\cdot,\tau)$*, if there is always a box $\mathbf{b}'$ in the box pruning output satisfying* $\text{IOA}(\mathbf{b}_{gt}, \mathbf{b}') > T$*, regardless of the rest of box pruning inputs $\mathcal{B}_{mask} \setminus \{\mathbf{b}_m\}, \mathcal{B}_{base}$. Formally, we have:* $\forall\ \mathcal{B}_{mask}$ s.t. $\mathbf{b}_m \in \mathcal{B}_{mask}$, $\forall\ \mathcal{B}_{base}$, $\exists\ \mathbf{b}' \in \mathcal{B}_{robust} =$ IOA-BOXPRUNE$(\mathcal{B}_{mask},\ \mathcal{B}_{base}, \tau)$ s.t. $\text{IOA}(\mathbf{b}_{gt}, \mathbf{b}') > T$.

We note that this definition considers all possible $\mathcal{B}_{\text{mask}}$ that contain $\mathbf{b}_{\text{m}}$ and all possible $\mathcal{B}_{\text{base}}$. This captures adaptive attackers' capability to maliciously manipulate some of the masked boxes in $\mathcal{B}_{\text{mask}}$ (when the patch is not removed by the masks) and all base boxes in $\mathcal{B}_{\text{base}}$ to interfere with the box pruning procedure. With Definition 3, we can discuss a sufficient condition of certifiable robustness in Lemma 1.

Lemma 1. *Given a ground-truth box $\mathbf{b}_{gt}$, one patch region $\mathbf{r}$, if we detect a pruning-safe masked box $\mathbf{b_m}$ from a masked image $\mathbf{x} \odot \mathbf{m}$ with no adversarial pixels (i.e., $\mathbf{m}[i,j] \leq \mathbf{r}[i,j], \forall (i,j)$),* IOA-OBJECTSEEKER *has certifiable IoA robustness for object $\mathbf{b}_{gt}$ against attacker $\mathcal{A}_{\{\mathbf{r}\}}$.*

Proof. Since the pruning-safe masked box $\mathbf{b_m}$ is detected from a masked image without adversarial pixels, we have $\mathbf{b_m} \in \mathcal{B}_{\text{mask}}$ no matter what a patch attacker does using $\mathbf{r}$. From the definition of pruning-safe mask box, we have the guarantee that $\exists\ \mathbf{b}' \in \mathcal{B}_{\text{robust}}$ s.t. $\text{IOA}(\mathbf{b}_{\text{gt}}, \mathbf{b}') > T$, which implies certifiable IoA robustness. □

With Lemma 1, an IoA robustness certification procedure only needs to look for pruning-safe masked boxes detected on masked images without adversarial pixels. Next, we will present two lemmas discussing how to identify pruning-safe masked boxes.

Recall that the box pruning involves two steps of box filtering (Line 20 of Algorithm 1) and box unionizing (Line 21 of Algorithm 1). Lemma 2 will give a lower bound of the IoA guarantee for the first step (box filtering). Lemma 3 will use this lower bound to further derive the sufficient condition of being a pruning-safe masked box for the entire box pruning procedure (box filtering and box unionizing).

Lemma 2. *Given any ground-truth object box $\mathbf{b}_{gt}$, if a detected masked box $\mathbf{b}_m$ is filtered by a box $\mathbf{b}_b$ during box filtering, i.e., $\text{IOA}(\mathbf{b}_m, \mathbf{b}_b) > \tau$, we have:*

$$\text{IOA}(\mathbf{b}_{gt}, \mathbf{b}_b) > \frac{|\mathbf{b}_m| \cdot \tau - |\mathbf{b}_m \setminus \mathbf{b}_{gt}|}{|\mathbf{b}_{gt}|},\ \forall\ \mathbf{b}_b \ \text{ s.t. } \ \text{IOA}(\mathbf{b}_m, \mathbf{b}_b) > \tau$$

Proof. From $\text{IOA}(\mathbf{b_m}, \mathbf{b_b}) = |\mathbf{b_m} \cap \mathbf{b_b}|/|\mathbf{b_m}| > \tau$, we have $|\mathbf{b_m} \cap \mathbf{b_b}| > |\mathbf{b_m}| \cdot \tau$. With this condition, we can derive an inequality as follows: $|\mathbf{b}_{\text{gt}} \cap \mathbf{b_b}| = |(\mathbf{b}_{\text{gt}} \cap \mathbf{b_b}) \cap \mathbf{b_m}| + |(\mathbf{b}_{\text{gt}} \cap \mathbf{b_b}) \setminus \mathbf{b_m}| \geq |\mathbf{b_m} \cap \mathbf{b_b} \cap \mathbf{b}_{\text{gt}}| = |\mathbf{b_m} \cap \mathbf{b_b}| - |(\mathbf{b_m} \cap \mathbf{b_b}) \setminus \mathbf{b}_{\text{gt}}| \geq |\mathbf{b_m} \cap \mathbf{b_b}| - |\mathbf{b_m} \setminus \mathbf{b}_{\text{gt}}| > \mathbf{b_m} \cdot \tau - |\mathbf{b_m} \setminus \mathbf{b}_{\text{gt}}|$ (two equal signs are based on basic set operations). Finally, we have $\text{IOA}(\mathbf{b}_{\text{gt}}, \mathbf{b_b}) = |\mathbf{b}_{\text{gt}} \cap \mathbf{b_b}|/|\mathbf{b}_{\text{gt}}| > (|\mathbf{b_m}| \cdot \tau - |\mathbf{b_m} \setminus \mathbf{b}_{\text{gt}}|)/|\mathbf{b}_{\text{gt}}|$. □

We use $\mathbb{L}_{\text{IOA}}(\mathbf{b}_{\text{gt}}, \mathbf{b_m}, \tau) = (|\mathbf{b_m}| \cdot \tau - |\mathbf{b_m} \setminus \mathbf{b}_{\text{gt}}|)/|\mathbf{b}_{\text{gt}}|$ to denote the lower bound. Lemma 3 will demonstrate that $\mathbb{L}_{\text{IOA}} > T$ implies that $\mathbf{b_m}$ is a pruning-safe masked box.

Lemma 3. *Given a ground-truth object $\mathbf{b}_{gt}$ and the box filtering threshold τ, if there is one masked box $\mathbf{b}_m \in \mathcal{B}_{mask}$ satisfying that $\mathbb{L}_{\text{IOA}}(\mathbf{b}_{gt}, \mathbf{b}_m, \tau) > T$, this box is a pruning-safe masked box for object $\mathbf{b}_{gt}$ and* IOA-BOXPRUNE$(\cdot,\cdot,\tau)$.

Proof. The detected masked box $\mathbf{b_m} \in \mathcal{B}_{\text{mask}}$ will go through box filtering and box unionizing to generate the final output. We will demonstrate that this masked box $\mathbf{b_m}$ will "survive" these two operations and become part of the final output, ensuring the IoA robustness.

Box filtering. Recall that, in Line 20 of Algorithm 1, we remove a box $\mathbf{b_m}$ if there is another box $\mathbf{b_b} \in \mathcal{B}_{\text{base}}$ satisfying $\text{IOA}(\mathbf{b_m}, \mathbf{b_b}) > \tau$, and the masked box set $\mathcal{B}_{\text{mask}}$ becomes $\mathcal{B}_{\text{mask}}^{\text{filtered}}$ after the filtering. We can prove that there is always a box $\mathbf{b}^* \in \mathcal{B}_{\text{mask}}^{\text{filtered}} \cup \mathcal{B}_{\text{base}}$ such that $\text{IOA}(\mathbf{b}_{\text{gt}}, \mathbf{b}^*) > T$. There are two possible scenarios.

1) If $\mathbf{b_m}$ is not filtered, we will know there is a box $\mathbf{b}^* = \mathbf{b_m} \in \mathcal{B}_{\text{mask}}^{\text{filtered}}$ such that $\text{IOA}(\mathbf{b}_{\text{gt}}, \mathbf{b}^*) = |\mathbf{b}_{\text{gt}} \cap \mathbf{b_m}|/|\mathbf{b}_{\text{gt}}| = (|\mathbf{b_m}| - |\mathbf{b_m} \setminus \mathbf{b}_{\text{gt}}|)/|\mathbf{b}_{\text{gt}}| \geq \mathbb{L}_{\text{IOA}}(\mathbf{b}_{\text{gt}}, \mathbf{b_m}, \tau) > T$.
2) If $\mathbf{b_m}$ is filtered, there is a box $\mathbf{b}^* \in \mathcal{B}_{\text{base}}$ such that $\text{IOA}(\mathbf{b_m}, \mathbf{b}^*) > \tau$. From Lemma 2, we know that this box $\mathbf{b}^* \in \mathcal{B}_{\text{base}}$ satisfies $\text{IOA}(\mathbf{b}_{\text{gt}}, \mathbf{b}^*) > \mathbb{L}_{\text{IOA}}(\mathbf{b}_{\text{gt}}, \mathbf{b_m}, \tau) > T$.

Box unionizing. Recall that we will perform clustering over filtered masked boxes $\mathcal{B}_{\text{mask}}^{\text{filtered}}$ and take the mathematical union of each cluster of boxes as the representative of each cluster to get $\mathcal{B}_{\text{mask}}^{\text{pruned}}$ (Line 21 of Algorithm 1). Since the mathematical union operation will not decrease IoA, we know that there is a box $\mathbf{b}' \in \mathcal{B}_{\text{mask}}^{\text{pruned}} \cup \mathcal{B}_{\text{base}} = \mathcal{B}_{\text{robust}}$ such that $\text{IoA}(\mathbf{b}_{\text{gt}}, \mathbf{b}') \geq \text{IoA}(\mathbf{b}_{\text{gt}}, \mathbf{b}^*) > T$, which implies the certifiable robustness for the object $\mathbf{b}_{\text{gt}}$. □

Certification. Lemma 1 and Lemma 3 together provide a simple way to certify IoA robustness: if we can detect a masked box $\mathbf{b}_{\text{m}}$ on a masked image with no adversarial pixel ($\mathbf{m}[i,j] \leq \mathbf{r}[i,j], \forall(i,j)$), and this box is pruning-safe ($\mathbb{L}_{\text{IoA}} > T$), we have certifiable IoA robustness for the object $\mathbf{b}_{\text{gt}}$ against the patch region $\mathbf{r}$. Recall that Algorithm 2 only returns `True` when these conditions are satisfied for all possible patch regions $\mathbf{r} \in \mathcal{R}$ (e.g., patches at different locations). Therefore, our certification in Algorithm 2 has accounted for all possible attackers within $\mathcal{A}_{\mathcal{R}}$. □

Remark: usage of Algorithm 2. Algorithm 2 and Theorem 1 allow us to determine the certifiable robustness of ObjectSeeker for a ground-truth object $\mathbf{b}_{\text{gt}}$ in a given image $\mathbf{x}$ against a given threat model $\mathcal{A}_{\mathcal{R}}$. In our evaluation, we will report the fraction of certified objects in the annotated test set of benchmark datasets as the robustness metric. We note that the certification procedure (Algorithm 2) is only used for robustness *evaluation* and thus requires ground-truth annotations and specific patch information. When we deploy ObjectSeeker in the wild, we use the inference procedure (Algorithm 1) instead and thus do not need any ground-truth annotation or patch information.

4. Evaluation

In this section, we implement ObjectSeeker with two vanilla object detectors and evaluate the defense performance on two object detection datasets (we include a third dataset in our technical report [28]). We demonstrate a significant robustness improvement (∼10%-40% absolute and ∼2-6× relative) over the prior work DetectorGuard [13] as well as similarly high clean performance (∼1% drop compared with vanilla undefended models).

4.1. Setup

In this subsection, we introduce our evaluation setup, including datasets, object detectors, evaluation metrics, robustness evaluation setup, and defense setup.

Datasets.

VOC [14]. The detection challenge of the PASCAL Visual Object Classes (VOC) project has annotations for 20 different object classes. We combine the `trainval2007` set (5k images) and the `trainval2012` set (11k images) for training and evaluate ObjectSeeker on the `test2007` set (5k images), which is a conventional usage of the PASCAL VOC dataset [29], [30].

COCO [15]. The Microsoft Common Objects in COntext (COCO) dataset is a challenging object detection dataset with 80 annotated object classes. We use the `COCO2017` split for training (117k images) and validation (5k images).

Object detectors.

YOLOR [16] is a popular one-stage object detector that achieves a good balance between inference speed and accuracy. We choose YOLOR-S [16] as the base object detector in ObjectSeeker.

Swin Transformer [18] adopts the representative two-stage detector Mask R-CNN [17] architecture and uses the Swin Transformer [18] as the backbone. Its largest model achieves state-of-the-art detection performance on COCO. We choose Swin-S for our experiments.

Evaluation metrics.

Clean performance: Average Precision (AP). We use AP as our evaluation metric for clean performance, which follows object detection benchmark competitions [14], [15] and relevant research papers [31], [32], [21], [22], [17], [33], [34], [29], [17], [16]. An object detector will have different precision and recall values as we change its confidence threshold γ (recall that $\mathbb{F}(\mathbf{x}, \gamma)$ only outputs boxes with confidence values larger than γ). AP is defined as the average of precision values at different recalls. Intuitively, AP considers model performance at different confidence thresholds, precision values, and recall values; thus, it provides a view of the model's overall performance. We note that AP is high only when *both precision and recall* are high. In our evaluation, we report $\text{AP}_{0.5}$ (AP evaluated with an IoU threshold of 0.5).

Robustness performance: Certified Recall (CertR). We use *certified recall* to evaluate defense robustness against patch hiding attacks. The certified recall is defined as the fraction of ground-truth *objects* whose IoA robustness can be certified by our defense (for which Algorithm 2 returns `True`). We note that the certified recall changes as the *clean* recall of the object detector changes (when we use a different confidence threshold γ). To enable a fair comparison, we report certified recall at a particular clean recall (CertR@0.x). We report CertR@0.8 for VOC and CertR@0.6 for COCO, which follows DetectorGuard [13].

Robustness evaluation setup. To evaluate the certifiable robustness of ObjectSeeker and to fairly compare with DetectorGuard [13], we choose a square patch that takes 1% of the image pixels and report certified recalls for a certification threshold $T = 0$. We will also analyze defense performance when we use different patch sizes (e.g., 1-100% pixels), patch shapes (e.g., rectangles), and large certification thresholds T (e.g., 0-0.9) in Section 4.3 and 4.4.

As discussed in Section 2.2, we consider three location models. We consider a patch location as a far-patch when the smallest distance along the height (and width) axis between any adversarial pixel and object pixel is larger than 10% of the image height (and width). We count an over-patch when there are adversarial pixels within the object bounding box. We consider the remaining patch locations as close-patch. We find that certified robustness against *all possible locations* is identical to that for over-patch locations, implying that over-patches are the hardest cases for defenders.

TABLE 2. PERFORMANCE OF VANILLA UNDEFENDED MODELS, OBJECTSEEKER, AND DETECTORGUARD [13]

	Dataset		PASCAL VOC [14]					MS COCO [15]				
	Model \ Metric	Certify class?	$AP_{0.5}$	FAR	Certified recall (@0.8) far-patch	close-patch	over-patch	$AP_{0.5}$	FAR	Certified recall (@0.6) far-patch	close-patch	over-patch
YOLOR [16]	Vanilla (undefended)	–	94.3%	–	–	–	–	70.3%	–	–	–	–
	ObjectSeeker	✓	92.9%	–	58.9%	46.7%	18.0%	69.3%	–	41.5%	28.8%	15.5%
	ObjectSeeker	✗	93.2%	–	61.9%	49.8%	21.5%	69.8%	–	44.6%	31.8%	18.1%
	DetectorGuard [13]	✗	93.0%	5.2%	32.7%	25.1%	12.0%	69.6%	2.4%	13.7%	8.0%	3.0%
Swin [18]	Vanilla (undefended)	–	93.9%	–	–	–	–	69.6%	–	–	–	–
	ObjectSeeker	✓	92.6%	–	68.0%	55.2%	22.5%	68.9%	–	34.9%	24.8%	11.7%
	ObjectSeeker	✗	92.9%	–	70.8%	58.0%	26.9%	69.2%	–	37.5%	27.4%	14.2%
	DetectorGuard [13]	✗	92.8%	3.6%	31.2%	23.0%	10.4%	69.2%	1.4%	11.4%	6.8%	2.4%

Note: the patch-agnostic property. We note that the setup of ObjectSeeker is agnostic to the shape, size, and location of the adversarial patch; the specified patch information is only used for robustness evaluation/certification. In other words, our defense algorithm and setup do not change when we consider a different patch shape, size, or location, though the robustness performance can change for different patches.

Defense setup. In our default setting, we set the number of vertical/horizontal lines $k = 30$ and the filtering threshold $\tau = 0.6$. We use different confidence thresholds $\gamma_{\text{b}}, \gamma_{\text{m}}$ for base boxes and masked boxes to adjust the *clean recall* of ObjectSeeker for AP and CertR@0.x evaluation; we provide additional details in Appendix A. We note that we choose these parameters because they can give similarly small clean performance drops ($\sim$1%) compared with undefended models on a validation set. In our technical report [28], we further demonstrate that using different randomly selected validation sets gives identical parameter selection results, and thus our default parameters are not "overfitted" to the evaluation setup of this section. In Section 4.3, We will further analyze the impact of different defense parameters.

We will also evaluate the performance of DetectorGuard [13] using their official open-source code. We use YOLOR and Swin as its vanilla object detectors for a fair performance comparison. We further report false alert rate (FAR) for DetectorGuard since it is an attack-detection defense. FAR is the fraction of clean images for which DetectorGuard issues a false alert. Note that ObjectSeeker is alert-free so it always has a zero FAR.

4.2. State-of-the-art Performance of ObjectSeeker

In Table 2, we report the defense performance of ObjectSeeker and compare it with DetectorGuard [13]. We note that ObjectSeeker is able to certify the correct class label of the detected box (in addition to detecting the object); in contrast, DetectorGuard has no guarantee for the label. To enable a fair comparison, we additionally report performance for a variant of ObjectSeeker that ignores the class labels in the step of secure box pruning. We also include the AP metric of vanilla undefended models to understand the clean performance of defenses.

ObjectSeeker achieves high certified recall across different datasets and threat models. Table 2 demonstrates that ObjectSeeker achieves high certified recalls. For example, ObjectSeeker-Swin (certify class) achieves a 68.0% certified recall for the far-patch model on the VOC dataset. That is, for 68.0% of the objects in the test set of the VOC dataset, no patch hiding attacker using a 1%-pixel square patch that is far away from the object can bypass our defense (i.e., hide the object). We can also see similarly high numbers across different object detectors, datasets, and threat models.

ObjectSeeker has a similarly high clean performance as vanilla object detectors. Comparing APs of vanilla models and ObjectSeeker in Table 2, we can see that ObjectSeeker achieves a similar clean AP as the vanilla object detectors – the AP drops are only $\sim$1%. The high clean performance can foster the real-world deployment of our defense. We note that the clean performance of ObjectSeeker that certifies class labels is slightly worse than that without class certification. This is because the vanilla object detector can make mistakes between similar object classes (e.g., motorbike vs. bicycle, car vs. bus) when the object is partially masked. Despite the small clean AP drop, we note that ObjectSeeker achieves a stronger robustness notion by certifying class labels.

ObjectSeeker achieves significant performance improvements compared with DetectorGuard [13]. We also compare defense performance between ObjectSeeker and DetectorGuard [13]. First, we can see that ObjectSeeker achieves a significant improvement in certified recalls. For example, ObjectSeeker-Swin (not certify class) improves the certified recall by 2$\times$ across three different location models on VOC (16.5%-39.6% absolute CertR improvements). The relative improvement on COCO is even larger: ObjectSeeker-YOLOR improves the certified recall by 6$\times$ against the over-patch attacker. We note the significant improvement holds even when we require ObjectSeeker to certify class label: we have achieved much higher certified recalls for an even stronger robustness notion. All these results demonstrate the strength of ObjectSeeker.

Second, ObjectSeeker also has similarly high clean performance as DetectorGuard. When we do not certify class label, ObjectSeeker has slightly higher clean APs than DetectorGuard. When we require ObjectSeeker to protect class labels, the clean APs are slightly lower. Moreover, we want to note that DetectorGuard is an attack-detection defense: it alerts and abstains from making predictions when it detects

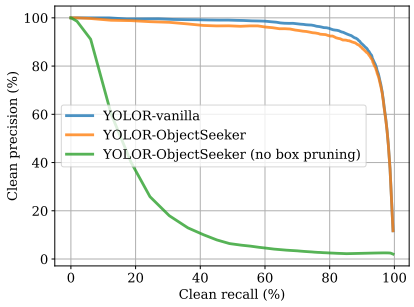

Figure 3. Clean precision vs. clean recall

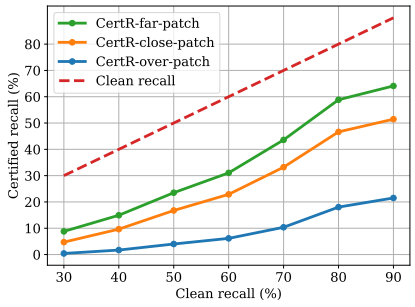

Figure 4. Certified recall vs. clean recall

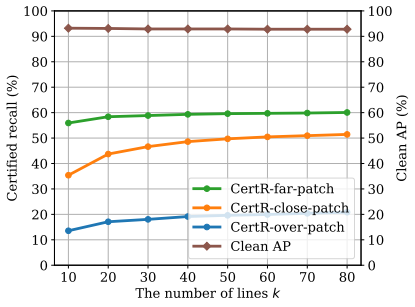

Figure 5. Effect of the number of lines k

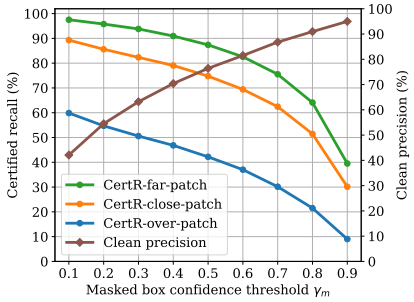

Figure 6. Effect of masked box confidence threshold γ_{m}

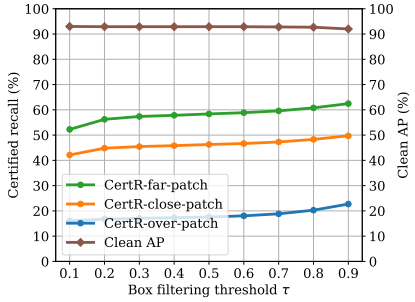

Figure 7. Effect of box filtering threshold τ

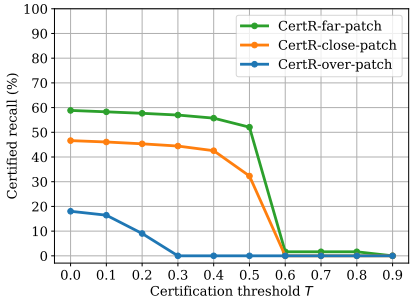

Figure 8. ObjectSeeker performance for different certification thresholds T

an attack. This design can cause non-trivial false alerts on the clean images and downgrade the user experience. In contrast, ObjectSeeker is an alert-free defense and thus has more advantages in real-world deployment.

Summary. In this subsection, we demonstrate that ObjectSeeker achieves high certified recalls while maintaining high clean APs as vanilla undefended models. Through a comparison with the only prior work DetectorGuard [13], we further demonstrate ObjectSeeker's state-of-the-art defense performance against patch hiding attacks.

4.3. Detailed Analysis of ObjectSeeker

In this subsection, we perform detailed analyses using the YOLOR detector and the VOC dataset. We will report defense performance at different clean recalls, discuss the effect of different defense parameters, and analyze the defense overhead.

Clean precision vs. clean recall. In Figure 3, we plot the clean precision-recall curves for vanilla YOLOR and ObjectSeeker-YOLOR. First, we can see that two curves are close to each other, explaining similar clean APs reported in Table 2. Second, ObjectSeeker-YOLOR has slightly lower precision than vanilla YOLOR when the precision value is higher than 85%; this explains the slight AP drop in Table 2. Third, we additionally plot the curve for ObjectSeeker without the secure box pruning module. We can see that this variant has a very low precision, which demonstrates the necessity and effectiveness of our box pruning module. Finally, we note that the precision of both vanilla YOLOR and ObjectSeeker-YOLOR starts to drop quickly as the clean recall increases over 80%. Therefore, we choose to study model robustness at a clean recall of 0.8 (CertR@0.8) for VOC in Table 2, when the model has both high clean precision and high clean recall.

Certified recall vs. clean recall. In Figure 4, we report certified recall at different clean recall values (recall that we only report CertR@0.8 in Table 2). As shown in the figure, the certified recall increases as clean recall increases. This is expected since the more objects we can detect in the clean setting, the more objects we can try to provide certifiable robustness for. Moreover, we note that the gap between clean recall and certified recall is stable as we vary the clean recall values. This implies that ObjectSeeker is compatible with detectors at different clean recall levels. How to further close this gap is an important future research question.

Effect of the number of lines k. In this analysis, we vary the number of lines k used for the mask set generation, and plot the defense performance in Figure 5. As shown in the figure, when we use a larger k, the robustness (CertR) gradually improves because we have a finer granularity of masks to bound the corrupted image region (recall Figure 2 in Section 3.1). Meanwhile, we can see a slight drop ($<0.5\%$) in clean AP as we increase k. This is because we have more masked images and more masked boxes, which leads to some unsuccessfully pruned boxes and hurts the precision of object detection. Furthermore, we find that the gain in certified robustness becomes minimal when k is larger than 30. Therefore, we choose $k = 30$ in our default setting to avoid excessive computational overhead.

Effect of the masked box confidence threshold γ_{m}.

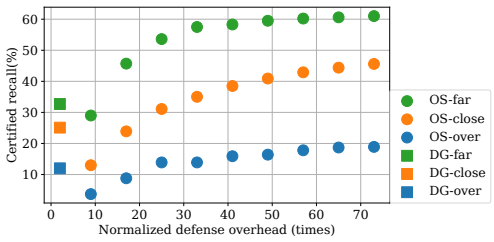

Figure 9. Trade-off between defense overhead and certified robustness (OS: ObjectSeeker; DG: DetectorGuard [13])

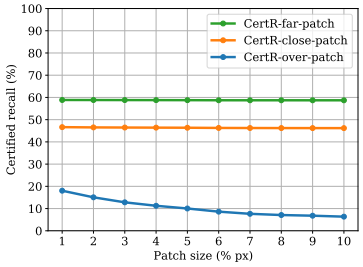

Figure 10. Robustness against different patch sizes ($k = 30$)

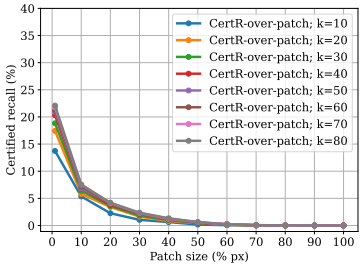

Figure 11. Robustness against different over-patch sizes with different k

In Figure 6, we study the effect of masked box confidence threshold γ_{m} (with a fixed γ_{b}). Recall that we only consider masked boxes whose prediction confidence exceeds the threshold γ_{m}. Figure 6 demonstrates that the threshold γ_{m} greatly affects the clean precision and the certified recall of ObjectSeeker. When we use a small threshold γ_{m}, we have more masked boxes and have a better chance for successful robustness certification. However, a lower threshold leads to less confident and less precise predictions, which results in more incorrect boxes and decreases the clean precision.

Effect of box filtering threshold τ. In Figure 7, we analyze the effect of box filtering threshold τ. As we use a smaller τ, the AP of ObjectSeeker improves since there are fewer boxes left after the box filtering operation. However, the certified robustness is downgraded with a smaller τ (we note that CertR drops are larger if we consider a larger T; see Appendix D for more results). This is because the lower bound proved in Lemma 2 gets lower with a smaller τ, making the robustness certification harder to succeed.

ObjectSeeker performance for different certification thresholds T. In our default setting, we set the certification threshold $T = 0$. That is, we consider the defense is robust if we can detect even just a tiny part of the object. This setup follows DetectorGuard [13] and enables a fair comparison in Table 2. Here, we study the defense performance when we require the defense to detect at least T of the object. We report the results in Figure 8. We can see that the CertR for over-patch is greatly affected by the certification threshold T. This is because we allow the adversary to place a patch at any location over the object. A worst-case attacker can put the patch at the center of the object to minimize the IoA guarantee (sometimes even occluding the entire small objects). Furthermore, we can see that the robustness for close-patch and far-patch are generally stable until the threshold T hits a large value. This demonstrates that ObjectSeeker provides stronger robustness when the patch does not overlap with objects. In Appendix D, we further discuss the practical implications of different T.

Defense overhead. In Figure 9, we plot the defense overheads (normalized by the runtime of the base vanilla object detector) versus certified recalls for DetectorGuard [13] and ObjectSeeker. As shown in the figure, when the computational budget is small (lower than 20×), DetectorGuard has better robustness (with a small overhead of 2×). However, as we have more computational resources, ObjectSeeker's performance gradually improves and eventually outperforms DetectorGuard by a large margin. This trade-off between defense overhead and defense performance should be carefully balanced when deploying the ObjectSeeker defense. Moreover, we note that our approach can be trivially parallelized with multiple GPUs (performing vanilla predictions for different masked images simultaneously). In Appendix D, we provide a quantitative analysis of absolute wall-clock runtime. We will show that using 8 GPUs can give 6.6× speedup. Together with other implementation-level optimizations, we can run ObjectSeeker on VOC with a latency of 40ms (25fps).

4.4. Different Patch Sizes and Shapes

In this subsection, we analyze defense performance against different patch sizes and shapes. Note that the performance is evaluated using the same defense setup (recall the patch-agnostic property).

ObjectSeeker performance against different patch sizes. In Figure 10, we plot the defense performance against patches with different sizes (from 1% to 10% image pixels). As we use a large patch size, the robustness against the over-patch model gradually drops. This is expected since a larger patch has a greater chance to occlude the salient part of the object. Intriguingly, we find that the robustness for close-patch and far-patch exhibits a different behavior: the certified recalls barely change when faced with a larger patch. This analysis further demonstrates that our defense has constrained the adversarial effect to a local region: it is harder for the attacker to hide objects that do not overlap with the patch. This property of ObjectSeeker can be helpful in practice when the attacker is not always able to place the patch over the victim object.

Furthermore, we report CertR for over-patch whose size ranges from 1-100% when we use different k in Figure 11. We can see that the CertR curves for different k are highly similar across different patch sizes. This further demonstrates the patch-agnostic property of ObjectSeeker: the default $k = 30$ used in the paper is not implicitly optimized for small patches (e.g., occupying 1% image pixels).

ObjectSeeker performance against different patch shapes. In this analysis, we study the defense performance against different patch shapes. In Table 3, we report certified recalls for different rectangular shapes that take up 1%

TABLE 3. CERTIFIED RECALLS (%) AGAINST ONE 1%-PIXEL PATCH OF DIFFERENT RECTANGLE SHAPES

aspect ratio	16:1	8:1	4:1	2:1	1:1	1:2	1:4	1:8	1:16
far-patch	58.8	58.8	58.9	58.9	58.9	58.9	58.9	58.9	58.9
close-patch	46.7	46.7	46.7	46.7	46.7	46.7	46.6	46.7	46.7
over-patch	21.3	19.2	19.6	18.8	18.0	18.1	16.8	18.3	18.3

TABLE 4. DEFENSE PERFORMANCE AGAINST TWO 0.5%-PIXEL SQUARE PATCHES FOR 200 VOC TEST IMAGES

Model \ Metric	$AP_{0.5}$	Certified recall (@0.8)		
		far-patch	close-patch	over-patch
YOLOR	94.4%	–	–	–
ObjectSeeker-YOLOR	94.4%	51.7%	24.6%	5.1%

of the image pixels (the aspect ratio ranging from 16:1 to 1:16). The results demonstrate that ObjectSeeker is effective against different patch shapes: CertRs for far-patch and close-patch barely change; CertRs for over-patch only change slightly. We note that these results are obtained *with the same set of defense parameters*, which further validates the patch-agnostic property of our masking defense.

Furthermore, we note that robustness certified in Table 3 directly applies to other shapes that can be completely covered by the rectangles. For example, since a 1%-pixel square can cover a $(\pi/4)$%-pixel circle, certified robustness for a 1%-pixel square also holds for a $(\pi/4)$%-pixel circle.

5. Discussion

In this section, we discuss the limitations and future work directions of ObjectSeeker.

Robustness against multiple patches. In this paper, we focus on the setting of *one* adversarial patch with unknown content, shape, size, and location because it is an unresolved research question. However, the design of ObjectSeeker is general: we only require that some masks from the mask set $\mathcal{M}$ can remove all adversarial pixels. If we have a new mask set that can mask out all (multiple) patches, we can simply plug this new mask set $\mathcal{M}$ into ObjectSeeker.

To provide a proof-of-concept, we experiment with two 0.5%-pixel patches on the VOC dataset. First, we generate a mask set $\mathcal{M}'$ (for one patch) as discussed in Section 3.1. Second, we generate a new mask set $\mathcal{M}$ that contains all possible two-mask combinations from the mask set $\mathcal{M}'$. Formally, we have $\mathcal{M} = \{\mathbf{m}_0 \odot \mathbf{m}_1 | (\mathbf{m}_0, \mathbf{m}_1) \in \mathcal{M}' \times \mathcal{M}'\}$. Third, we use $\mathcal{M}$ to instantiate ObjectSeeker. We report the defense performance in Table 4. We can see that our defense has high clean performance and achieves non-trivial certified recall against two-patch attacks.

Improving ObjectSeeker runtime. ObjectSeeker needs to perform vanilla object detection on $4k$ masked images. As analyzed in Figure 5 and Figure 9, k needs to be large enough to achieve good defense robustness. As a result, the ObjectSeeker defense incurs a non-negligible overhead. We note that it is worthwhile to spend more computation for high certifiable robustness for applications whose robustness is important. For example, for the security-critical video content analysis, we can apply ObjectSeeker to offline video to ensure robustness. Nevertheless, it is also important to study how to reduce defense overhead with both algorithm and implementation-level improvements. An algorithm-level optimization for real-time systems could be applying ObjectSeeker to a subset of frames to balance the efficiency and robustness. Implementation-level optimizations could include parallelizing inference on masked images with multiple GPUs and resizing input images; we provide quantitative examples in Appendix D.

Improving robustness against over-patch. In our evaluation, we follow DetectorGuard [13] to use three patch location models to analyze ObjectSeeker against different attack capabilities. Despite the large *relative* improvements from DetectorGuard [13], we acknowledge that ObjectSeeker's *absolute* CertR for over-patch remains limited. Further enhancing robustness against the challenging over-patch attackers is an important future research objective.

Further exploration of similarity score functions $\mathbb{S}$ and robustness notions. In our ObjectSeeker design, we use a "robust" similarity score function $\mathbb{S}$ to prune redundant boxes, and we note that using different similarity scores $\mathbb{S}$ can provide different types of robustness notions. In this paper, we use IoA as the similarity score function $\mathbb{S}$ and focus on the concept of "IoA robustness" (i.e., $\text{IoA}(\mathbf{b}_{\text{gt}}, \mathbf{b}') = |\mathbf{b}_{\text{gt}} \cap \mathbf{b}'|/|\mathbf{b}_{\text{gt}}| > T$). This is because IoA captures how much of the ground-truth box is detected and aligns with the objective of defending against patch hiding attacks (following DetectorGuard [13]). In Appendix B, we discuss one alternative ObjectSeeker instance: we use IoU as $\mathbb{S}$ and achieve "IoU robustness" for far-patch attackers: we aim to predict a box $\mathbf{b}'$ for each ground-truth box $\mathbf{b}_{\text{gt}}$ such that $\text{IoU}(\mathbf{b}_{\text{gt}}, \mathbf{b}') = |\mathbf{b}_{\text{gt}} \cap \mathbf{b}'|/|\mathbf{b}_{\text{gt}} \cup \mathbf{b}'| > T$. We demonstrate that, against a far-patch attacker, we can achieve certified recall of $\sim$50% for VOC and $\sim$40% for COCO with an IoU certification threshold of 0.5. In Appendix C, we further provide a taxonomy of different robustness notions against patch hiding attacks.

Accounting for physical-world attack constraints. The physically realizable nature of patch attacks imposes a threat to real-world object detectors and motivates the design of our defense. In ObjectSeeker, however, we did not model physically realizable constraints such as printability and lighting conditions, but simply assumed that the attacker can introduce arbitrary pixel values. As a result, ObjectSeeker's certification is over-conservative against physical-world attacks. How to further leverage physical-world constraints and improve defense robustness and efficiency could also be an interesting future work direction.

6. Related Work

6.1. Adversarial Patch Attacks

Image classification. The adversarial patch attack was first introduced for *image classification*. Brown et al. [7] demonstrated that, by constraining all adversarial pixels within a local restricted region, an attacker can carry out the

patch attack in the physical world. The physically realizable nature of the patch attack imposed a huge threat to real-world computer vision systems. Follow-up papers further studied variants of patch attacks against image classifiers with different threat models [35], [36], [37], [38], [39].

Object detection. Numerous patch attacks against *object detection* have been proposed. Liu et al. [40] proposed the first patch attack against object detectors. Lu et al. [41], Chen et al. [42], Eykholt et al. [43], and Zhao et al. [1] proposed different physical attacks against traffic sign recognition. Thys et al. [2], Xu et al. [3], and Wu et al. [4] studied how to use adversarial patches to evade person detection. Recently, Hu et al. [5] and Tan et al. [6] propose natural-looking patch hiding attacks against object detectors.

6.2. Defenses against Adversarial Patches

Image classification. To counter the threat of adversarial patch attacks, there have been a large number of heuristic-based image classification defenses proposed in recent years [44], [45], [46], [47], [48]. Unfortunately, many of these defenses are found broken when there is an adaptive attacker who knows about the defense setup [12]. To provide a strong provable robustness guarantee for patch attacks, the research community has proposed a number of certifiably robust defenses [49], [50], [51], [23], [24], [52], [25], [26], [53], [54] for image classification models. In this paper, ObjectSeeker focuses on *a harder task of object detection*.

Object detection. How to secure object detectors is a challenging and under-studied research question. Saha et al. [8] studied how to constrain the use of spatial context in YOLOv2 [31] and improved the robustness against a patch at the image corner. Metzen et al. [9] studied meta-learning techniques to improve model robustness. Ji et al. [10] added adversarial patches to the training dataset and taught the model to detect patches. Liang et al [11] proposed two heuristic-based defenses to detect a patch hiding attack. Chiang et al. [12] designed a defense model to detect and remove the adversarial pixels. Despite their contributions to robust object detection research, these defenses are all based on heuristics and do not have any formal security guarantee.

In contrast, Xiang et al. [13] proposed DetectorGuard as the only certifiably robust defense against patch hiding attacks, which aimed to provide a robustness guarantee for certain certified objects against any adaptive attacker within the threat model. DetectorGuard designed an objectness explaining strategy to build certifiably robust object detectors using off-the-shelf certifiably robust image classifiers. However, DetectorGuard only achieved limited certified robustness (recall Table 2). Xiang et al. [13] pointed out that the bottleneck for the defense performance is the imperfection of existing certifiably robust image classifies: the incorrect classification outputs from the robust image classifier resulted in a heavy trade-off between robustness and clean performance. In this paper, we design ObjectSeeker solely based on vanilla undefended object detectors and easily bypass the bottleneck of imperfect image classifiers. In Section 4, we have demonstrated the significant improvements ($\sim$2-6$\times$) in robustness performance over DetectorGuard. Moreover, DetectorGuard is an attack-detection defense that has troublesome false alerts in the clean setting while we design ObjectSeeker as an alert-free defense. Finally, DetectorGuard cannot protect the class labels due to its design limitations while ObjectSeeker achieves high certified robustness for bounding box labels in addition to securing the class label (recall Table 2 in Section 4.2).

Masking-based defenses against adversarial patches. We note that the idea of masking out adversarial patches has been studied in existing defenses for image classification [44], [48], [23], [24], [25], [26] and object detection [12]. However, they either lack certifiable robustness [44], [48], [12], or require additional information on patch shapes/sizes [23], [24], [25], [26]. Our ObjectSeeker proposes the first patch-agnostic masking strategy that achieves certifiable robustness.

6.3. Other Adversarial Example Attacks and Defenses

Adversarial example attacks and defenses for computer vision tasks with different threat models have been extensively studied [55], [56], [57], [58], [59], [60], [61], [62], [63], [64], [65], [66], [67], [68], [69], [70], [71], [72]. We focus on the adversarial patch attacks because they are physically-realizable and impose an urgent threat to real-world computer vision systems.

7. Conclusion

In this paper, we propose ObjectSeeker as a certifiably robust defense against patch hiding attacks. ObjectSeeker is a two-step defense framework: we first perform patch-agnostic masking to neutralize the adversarial effect (without knowing patch size, shape, and location); we next perform secure box pruning for a precise and robust prediction output. We can certify that for certain objects, ObjectSeeker can always detect the object no matter what an adaptive attacker within the threat model does. Our certifiable evaluation demonstrates a significant improvement ($\sim$10%-40% absolute and $\sim$2-6$\times$ relative) in certified robustness over the only prior work DetectorGuard [13], as well as the high clean performance of ObjectSeeker ($\sim$1% drops compared with vanilla undefended models).

Acknowledgements

We are grateful to the anonymous shepherd and reviewers from IEEE S&P 2023 program committee for their insightful comments and helpful suggestions. We would like to thank Shawn Shan, Sihui Dai, and Vikash Sehwag for providing early feedback on the manuscript draft. This work was supported in part by the National Science Foundation under grant CNS-2131859, the ARL's Army Artificial Intelligence Innovation Institute (A2I2), Schmidt DataX award, and Princeton E-ffiliates Award.

References

[1] Y. Zhao, H. Zhu, R. Liang, Q. Shen, S. Zhang, and K. Chen, "Seeing isn't believing: Towards more robust adversarial attack against real world object detectors," in *CCS*, 2019.

[2] S. Thys, W. Van Ranst, and T. Goedemé, "Fooling automated surveillance cameras: adversarial patches to attack person detection," in *CVPR Workshop*, 2019.

[3] K. Xu, G. Zhang, S. Liu, Q. Fan, M. Sun, H. Chen, P. Chen, Y. Wang, and X. Lin, "Adversarial t-shirt! evading person detectors in a physical world," in *ECCV*, 2020.

[4] Z. Wu, S. Lim, L. S. Davis, and T. Goldstein, "Making an invisibility cloak: Real world adversarial attacks on object detectors," in *ECCV*, 2020.

[5] Y.-C.-T. Hu, B.-H. Kung, D. S. Tan, J.-C. Chen, K.-L. Hua, and W.-H. Cheng, "Naturalistic physical adversarial patch for object detectors," in *ICCV*, 2021.

[6] J. Tan, N. Ji, H. Xie, and X. Xiang, "Legitimate adversarial patches: Evading human eyes and detection models in the physical world," in *ACM International Conference on Multimedia*, 2021.

[7] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, "Adversarial patch," in *NeurIPS Workshop*, 2017.

[8] A. Saha, A. Subramanya, K. Patil, and H. Pirsiavash, "Role of spatial context in adversarial robustness for object detection," in *CVPR Workshop*, 2020.

[9] J. H. Metzen, N. Finnie, and R. Hutmacher, "Meta adversarial training against universal patches," in *ICML Workshop*, 2021.

[10] N. Ji, Y. Feng, H. Xie, X. Xiang, and N. Liu, "Adversarial yolo: Defense human detection patch attacks via detecting adversarial patches," *arXiv:2103.08860*, 2021.

[11] B. Liang, J. Li, and J. Huang, "We can always catch you: Detecting adversarial patched objects with or without signature," *arXiv:2106.05261*, 2021.

[12] P.-H. Chiang, C.-S. Chan, and S.-H. Wu, "Adversarial pixel masking: A defense against physical attacks for pre-trained object detectors," in *International Conference on Multimedia*, 2021.

[13] C. Xiang and P. Mittal, "Detectorguard: Provably securing object detectors against localized patch hiding attacks," in *ACM CCS*, 2021.

[14] M. Everingham, L. V. Gool, C. K. I. Williams, J. M. Winn, and A. Zisserman, "The pascal visual object classes (VOC) challenge," *International Journal of Computer Vision*, 2010.

[15] T. Lin, M. Maire, S. J. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: common objects in context," in *ECCV*, 2014.

[16] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "You only learn one representation: Unified network for multiple tasks," *arXiv:2105.04206*, 2021.

[17] K. He, G. Gkioxari, P. Dollár, and R. B. Girshick, "Mask R-CNN," in *ICCV*, 2017.

[18] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *ICCV*, 2021.

[19] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *The International Journal of Robotics Research*, 2013.

[20] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE CVPR*, 2016.

[21] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv:2004.10934*, 2020.

[22] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *NeurIPS*, 2015.

[23] M. McCoyd, W. Park, S. Chen, N. Shah, R. Roggenkemper, M. Hwang, J. X. Liu, and D. A. Wagner, "Minority reports defense: Defending against adversarial patches," in *ACNS Workshop*, 2020.

[24] C. Xiang, A. N. Bhagoji, V. Sehwag, and P. Mittal, "Patchguard: A provably robust defense against adversarial patches via small receptive fields and masking," in *USENIX Security*, 2021.

[25] C. Xiang and P. Mittal, "Patchguard++: Efficient provable attack detection against adversarial patches," in *ICLR Workshop*, 2021.

[26] C. Xiang, S. Mahloujifar, and P. Mittal, "Patchcleanser: Certifiably robust defense against adversarial patches for any image classifier," in *USENIX Security*, 2022.

[27] M. Ester, H. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *KDD*, 1996.

[28] C. Xiang, A. Valtchanov, S. Mahloujifar, and P. Mittal, "Objectseeker: Certifiably robust object detection against patch hiding attacks via patch-agnostic masking," *arXiv preprint arXiv:2202.01811*, 2022.

[29] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. E. Reed, C. Fu, and A. C. Berg, "SSD: single shot multibox detector," in *ECCV*, 2016.

[30] H. Zhang and J. Wang, "Towards adversarially robust object detection," in *ICCV*. IEEE, 2019.

[31] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *CVPR*, 2017.

[32] ——, "Yolov3: An incremental improvement," *arXiv:1804.02767*, 2018.

[33] T. Lin, P. Goyal, R. B. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *ICCV*. IEEE Computer Society, 2017.

[34] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *CVPR*, 2020.

[35] D. Karmon, D. Zoran, and Y. Goldberg, "LaVAN: Localized and visible adversarial noise," in *ICML*, 2018.

[36] C. Yang, A. Kortylewski, C. Xie, Y. Cao, and A. Yuille, "Patchattack: A black-box texture-based attack with reinforcement learning," in *ECCV*, 2020.

[37] A. Liu, X. Liu, J. Fan, Y. Ma, A. Zhang, H. Xie, and D. Tao, "Perceptual-sensitive GAN for generating adversarial patches," in *AAAI*, 2019.

[38] A. Liu, J. Wang, X. Liu, B. Cao, C. Zhang, and H. Yu, "Bias-based universal adversarial patch attack for automatic check-out," in *ECCV*, 2020.

[39] B. G. Doan, M. Xue, S. Ma, E. Abbasnejad, and D. C. Ranasinghe, "Tnt attacks! universal naturalistic adversarial patches against deep neural network systems," *arXiv:2111.09999*, 2021.

[40] X. Liu, H. Yang, Z. Liu, L. Song, Y. Chen, and H. Li, "DPATCH: an adversarial patch attack on object detectors," in *AAAI Workshop*, 2019.

[41] J. Lu, H. Sibai, and E. Fabry, "Adversarial examples that fool detectors," *arXiv:1712.02494*, 2017.

[42] S.-T. Chen, C. Cornelius, J. Martin, and D. H. P. Chau, "Shapeshifter: Robust physical adversarial attack on faster r-cnn object detector," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2018.

[43] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, F. Tramer, A. Prakash, T. Kohno, and D. Song, "Physical adversarial examples for object detectors," in *USENIX WOOT Workshop*, 2018.

[44] J. Hayes, "On visible adversarial perturbations & digital watermarking," in *CVPR Workshop*, 2018.

[45] M. Naseer, S. Khan, and F. Porikli, "Local gradients smoothing: Defense against localized adversarial attacks," in *WACV*, 2019.

[46] T. Wu, L. Tong, and Y. Vorobeychik, "Defending against physically realizable attacks on image classification," in *ICLR*, 2020.

[47] S. Rao, D. Stutz, and B. Schiele, "Adversarial training against location-optimized adversarial patches," in *ECCV Workshop*, 2020.

[48] N. Mu and D. Wagner, "Defending against adversarial patches with robust self-attention," in *ICML Workshop*, 2021.

[49] P.-Y. Chiang, R. Ni, A. Abdelkader, C. Zhu, C. Studor, and T. Goldstein, "Certified defenses for adversarial patches," in *ICLR*, 2020.

[50] Z. Zhang, B. Yuan, M. McCoyd, and D. Wagner, "Clipped bagnet: Defending against sticker attacks with clipped bag-of-features," in *Deep Learning and Security Workshop (DLS)*, 2020.

[51] A. Levine and S. Feizi, "(De)randomized smoothing for certifiable defense against patch attacks," in *NeurIPS*, 2020.

[52] J. H. Metzen and M. Yatsura, "Efficient certified defenses against patch attacks on image classifiers," in *ICLR*, 2021.

[53] H. Han, K. Xu, X. Hu, X. Chen, L. Liang, Z. Du, Q. Guo, Y. Wang, and Y. Chen, "Scalecert: Scalable certified defense against adversarial patches with sparse superficial layers," in *NeurIPS*, 2021.

[54] H. Salman, S. Jain, E. Wong, and A. Madry, "Certified patch robustness via smoothed vision transformers," in *CVPR*, 2022.

[55] M. Barreno, B. Nelson, A. D. Joseph, and J. D. Tygar, "The security of machine learning," *Machine Learning*, 2010.

[56] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *ICLR*, 2014.

[57] B. Biggio, I. Corona, D. Maiorca, B. Nelson, N. Šrndić, P. Laskov, G. Giacinto, and F. Roli, "Evasion attacks against machine learning at test time," in *ECML PKDD*, 2013.

[58] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *ICLR*, 2015.

[59] N. Papernot, P. D. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *EuroS&P*, 2016.

[60] D. Meng and H. Chen, "Magnet: A two-pronged defense against adversarial examples," in *CCS*, 2017.

[61] W. Xu, D. Evans, and Y. Qi, "Feature squeezing: Detecting adversarial examples in deep neural networks," in *NDSS*, 2018.

[62] N. Carlini and D. A. Wagner, "Towards evaluating the robustness of neural networks," in *IEEE S&P*, 2017.

[63] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *ICLR*, 2018.

[64] N. Papernot, P. D. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *IEEE S&P*, 2016.

[65] A. Raghunathan, J. Steinhardt, and P. Liang, "Certified defenses against adversarial examples," in *ICLR*, 2018.

[66] E. Wong and J. Z. Kolter, "Provable defenses against adversarial examples via the convex outer adversarial polytope," in *ICML*, 2018.

[67] M. Lécuyer, V. Atlidakis, R. Geambasu, D. Hsu, and S. Jana, "Certified robustness to adversarial examples with differential privacy," in *IEEE S&P*, 2019.

[68] J. M. Cohen, E. Rosenfeld, and J. Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *ICML*, 2019.

[69] H. Salman, J. Li, I. P. Razenshteyn, P. Zhang, H. Zhang, S. Bubeck, and G. Yang, "Provably robust deep learning via adversarially trained smoothed classifiers," in *NeurIPS*, 2019.

[70] S. Gowal, K. Dvijotham, R. Stanforth, R. Bunel, C. Qin, J. Uesato, R. Arandjelovic, T. A. Mann, and P. Kohli, "Scalable verified training for provably robust image classification," in *ICCV*, 2019.

[71] M. Mirman, T. Gehr, and M. T. Vechev, "Differentiable abstract interpretation for provably robust neural networks," in *ICML*, 2018.

[72] C. Xiang, C. R. Qi, and B. Li, "Generating 3d adversarial point clouds," in *CVPR*, 2019.

Appendix A.
Confidence Thresholds in ObjectSeeker

As presented in Algorithm 1, we have two separate confidence thresholds $\gamma_{\text{m}}, \gamma_{\text{b}}$ for masked boxes and base boxes. Here, we provide additional implementation details.

In our default setting, we set $\gamma_{\text{m}} = \max(\alpha, \gamma_{\text{b}} + (1 - \gamma_{\text{b}}) \cdot \beta), \alpha, \beta \in [0, 1]$. First, this ensures that low-quality and low-confidence (smaller than α) masked boxes will be discarded. Second, we will use a higher γ_{m} for e a higher γ_{b} to ensure high precision of ObjectSeeker. We note that different object detectors (e.g., YOLO vs. Faster RCNN) have different confidence value distributions for different datasets. In our experiments, we set $\alpha = 0.8, \beta = 0.8$ for YOLOR on VOC, $\alpha = 0.7, \beta = 0.5$ for YOLOR on COCO, $\alpha = 0.8, \beta = 0.5$ for Swin on VOC, and $\alpha = 0.9, \beta = 0.8$ for Swin on COCO.

To calculate AP for ObjectSeeker, we vary the base confidence threshold γ_{b} from 0 to 1 with a step of 0.01, change the γ_{m} correspondingly, and record the precision and recall values. Then we calculate the averaged precision at different recall values in the conventional way.

Finally, we note that clean recall can be considered as a universal normalizer of confidence threshold γ_{b}. Given that confidence value distributions of different detectors and datasets can be significantly different, we analyzed ObjectSeeker performance at "normalized" clean recalls instead of "raw" γ_{b} in Section 4.

Appendix B.
IoU as Similarity Score Function $\mathbb{S}$

In Section 3, we discussed the use of box similar score $\mathbb{S}$ and noted that different $\mathbb{S}$ can provide different robustness notions. In this section, we discuss a variant of ObjectSeeker that uses IoU as the similarity score $\mathbb{S}$ and can certify IoU robustness for the *far-patch model*.

The core idea of this ObjectSeeker variant is to split boxes detected on the masked image (masked boxes) into two groups and apply different pruning strategies (with different $\mathbb{S}$) to two groups. The first group contains the boxes that are far away from and do not overlap with the masks (termed as *non-overlapping boxes*); the second group considers the boxes that are close to or overlap with the masks (termed as *overlapping boxes*).

When the mask is far away from the given object, the detection of this object is barely affected. As a result, detected non-overlapping masked boxes are almost the same and thus have high pair-wise IoU with each other. We can then easily identify and prune redundant boxes by looking at the IoU. In Lemma 4 and Lemma 5, we can further prove that the IoU-based pruning strategy can provide a certifiable guarantee for IoU certification. After pruning the non-overlapping boxes, we use a similar IoA pruning strategy discussed in the main body to prune overlapping boxes. These boxes can be useful for certifying IoA robustness. We note that we cannot use IoU to prune overlapping boxes

Algorithm 3 ObjectSeeker inference algorithm with IoU

Input: Image $\mathbf{x}$, image size (W, H), base object detector $\mathbb{F}$, the number of lines k (for masking), masked boxes confidence threshold γ_{m}, base boxed confidence threshold γ_{b}, box pruning threshold $\tau_{\text{IoU}}, \tau_{\text{IoA}}$

Output: Robust detection results $\mathcal{B}_{\text{robust}}$

1: **procedure** OBJECTSEEKER
2: $\mathcal{M} \leftarrow \text{MASKSET}(k, W, H)$
3: $\mathcal{B}^{\text{no}}_{\text{mask}}, \mathcal{B}^{\text{o}}_{\text{mask}} \leftarrow \varnothing, \varnothing$
4: **for** $\mathbf{m} \in \mathcal{M}$ **do**
5: $\mathcal{B}^{\text{no}}_{\mathbf{m}}, \mathcal{B}^{\text{o}}_{\mathbf{m}} \leftarrow \text{SPLIT}(\mathbb{F}(\mathbf{x} \odot \mathbf{m}, \gamma_{\text{m}}), \mathbf{m})$
6: $\mathcal{B}^{\text{no}}_{\text{mask}}, \mathcal{B}^{\text{o}}_{\text{mask}} \leftarrow \mathcal{B}^{\text{no}}_{\text{mask}} \cup \mathcal{B}^{\text{no}}_{\mathbf{m}}, \mathcal{B}^{\text{o}}_{\text{mask}} \cup \mathcal{B}^{\text{o}}_{\mathbf{m}}$
7: **end for**
8: $\mathcal{B}_{\text{base}} \leftarrow \mathbb{F}(\mathbf{x}, \gamma_{\text{b}})$
9: $\mathcal{B}^{\text{no-filtered}}_{\text{mask}} \leftarrow \{\mathbf{b}_{\text{m}} \in \mathcal{B}^{\text{no}}_{\text{mask}} | \nexists\ \mathbf{b}_{\text{b}} \in \mathcal{B}_{\text{base}} : \text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau_{\text{IoU}}\}$
10: $\mathcal{B}^{\text{no-pruned}}_{\text{mask}} \leftarrow \text{NMS}(\mathcal{B}^{\text{no-filtered}}_{\text{mask}}, \tau_{\text{IoU}})$
11: $\mathcal{B}^{\text{o-filtered}}_{\text{mask}} \leftarrow \{\mathbf{b}_{\text{m}} \in \mathcal{B}^{\text{o}}_{\text{mask}} | \nexists\ \mathbf{b}_{\text{b}} \in \mathcal{B}_{\text{base}} : \text{IOA}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau_{\text{IoA}}\}$
12: $\mathcal{B}^{\text{o-pruned}}_{\text{mask}} \leftarrow \{\bigcup_{\hat{\mathbf{b}} \in \hat{\mathcal{B}}} \hat{\mathbf{b}} \mid \hat{\mathcal{B}} \in \text{DBSCAN}(\mathcal{B}^{\text{o-filtered}}_{\text{mask}})\}$
13: $\mathcal{B}_{\text{robust}} \leftarrow \mathcal{B}_{\text{base}} \cup \mathcal{B}^{\text{no-pruned}}_{\text{mask}} \cup \mathcal{B}^{\text{o-pruned}}_{\text{mask}}$
14: **return** $\mathcal{B}_{\text{robust}}$
15: **end procedure**

because a good number of overlapping boxes only detect part of the object. Therefore, their pair-wise IoU can be too low for effective pruning.

Pseudocode. We present the algorithm pseudocode in Algorithm 3. We first generate the mask set $\mathcal{M}$ and initialize two empty sets $\mathcal{B}^{\text{no}}_{\text{mask}}, \mathcal{B}^{\text{o}}_{\text{mask}}$ for holding non-overlapping and overlapping masked boxes (Line 3). Next, we iterate over every mask $\mathbf{m} \in \mathcal{M}$. For each mask, we perform object detection on the masked image $\mathbb{F}(\mathbf{x} \odot \mathbf{m}, \gamma_{\text{m}})$ and split the detected boxes into non-overlapping boxes $\mathcal{B}^{\text{no}}_{\mathbf{m}}$ and overlapping boxes $\mathcal{B}^{\text{o}}_{\mathbf{m}}$ (Line 5). We then add $\mathcal{B}^{\text{no}}_{\mathbf{m}}$ and $\mathcal{B}^{\text{o}}_{\mathbf{m}}$ to $\mathcal{B}^{\text{no}}_{\text{mask}}$ and $\mathcal{B}^{\text{o}}_{\text{mask}}$, respectively (Line 6). After gathering all masked boxes, we further get base boxes $\mathcal{B}_{\text{base}}$ prediction on the original image (Line 8) and perform secure box pruning on $\mathcal{B}^{\text{no}}_{\text{mask}}$ (Line 9-10) and $\mathcal{B}^{\text{o}}_{\text{mask}}$ (Line 11-12) separately. For non-overlapping boxes $\mathcal{B}^{\text{no}}_{\text{mask}}$, we first perform box filtering with IoU as the box similarity score $\mathbb{S}$ (Line 9). Next, we perform non-maximum suppression for box clustering and box representing (Line 10). The non-maximum suppression works as follows. First, it picks the box $\mathbf{b}_0$ with the highest confidence value, finds all boxes $\mathbf{b}_1$ satisfying $\text{IOU}(\mathbf{b}_0, \mathbf{b}_1) > \tau_{\text{IoU}}$, forming a cluster with these boxes. Second, it repeats the first step on remaining unclustered boxes until all boxes are clustered. Third, for each box cluster, it takes the box with the highest confidence as the representative. For overlapping boxes $\mathcal{B}^{\text{o}}_{\text{mask}}$, we perform box filtering (Line 11) and box unionizing (Line 12) with IoA, similar to what we discussed in Section 3.2.

Finally, we combine base boxes $\mathcal{B}_{\text{base}}$, pruned non-overlapping boxes $\mathcal{B}^{\text{no-pruned}}_{\text{mask}}$ and overlapping boxes $\mathcal{B}^{\text{o-pruned}}_{\text{mask}}$ together as the final prediction output $\mathcal{B}_{\text{robust}}$ (Line 13).

Certification. The certification with IoU robustness is similar to what we discussed for IoA in Section 3.3. From Line 9-10 of Algorithm 3, a masked box $\mathbf{b}_{\text{m}}$ will only be removed when there is another box $\mathbf{b}_{\text{b}}$ that has a large IoU with $\mathbf{b}_{\text{m}}$. Therefore, given a masked box $\mathbf{b}_{\text{m}}$ and a ground-truth box $\mathbf{b}_{\text{gt}}$, we only need to prove the lower bound of $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{b}})$ for any box $\mathbf{b}_{\text{b}}$ that can remove $\mathbf{b}_{\text{m}}$ (i.e., $\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau$).

Lemma 4. *For any ground-truth box $\mathbf{b}_{gt}$, detected masked box $\mathbf{b}_m$, and box $\mathbf{b}_b$ with $\text{IOU}(\mathbf{b}_m, \mathbf{b}_b) > \tau$, we have:*

$$\text{IOU}(\mathbf{b}_{gt}, \mathbf{b}_b) > \tfrac{\tau B + (\tau - 1) \cdot C}{A + B + C},\ \forall\ \mathbf{b}_b \ \text{ s.t. } \ \text{IOU}(\mathbf{b}_m, \mathbf{b}_b) > \tau$$

where $A = |\mathbf{b}_{gt} \setminus \mathbf{b}_m|, B = |\mathbf{b}_{gt} \cap \mathbf{b}_m|, C = |\mathbf{b}_m \setminus \mathbf{b}_{gt}|$.

Proof. In this proof, we are going to find the worst-case $\mathbf{b}_{\text{b}}$ that satisfies the condition of $\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau$ but gives the lowest $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{b}})$.

First, let us divide the box $\mathbf{b}_{\text{b}}$ into four disjoint parts using set operations.

$$\begin{aligned}\mathbf{b}_{\text{b}} =& (\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{gt}} \cup \mathbf{b}_{\text{m}})) \cup (\mathbf{b}_{\text{b}} \setminus (\mathbf{b}_{\text{gt}} \cup \mathbf{b}_{\text{m}})) \\ =& (\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{gt}} \cap \mathbf{b}_{\text{m}})) \cup (\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}})) \cup (\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{gt}} \setminus \mathbf{b}_{\text{m}})) \\ & \cup (\mathbf{b}_{\text{b}} \setminus (\mathbf{b}_{\text{gt}} \cup \mathbf{b}_{\text{m}}))\end{aligned}$$

To reason the worst-case $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{b}})$, we can set the second part $|\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}})|$ to $|(\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}})| = C$ because increasing its area increases $\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}})$ but not $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{b}})$. We set the third $|\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{gt}} \setminus \mathbf{b}_{\text{m}})| = 0$ because decreasing its area decreases $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{b}})$ but not $\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}})$.

Next, we set the remaining two parts as $|\mathbf{b}_{\text{b}} \cap (\mathbf{b}_{\text{gt}} \cap \mathbf{b}_{\text{m}})| = \alpha, |\mathbf{b}_{\text{b}} \setminus (\mathbf{b}_{\text{gt}} \cup \mathbf{b}_{\text{m}})| = \beta$. We can then write our constraint sets as

$$\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) = \frac{|\mathbf{b}_{\text{m}} \cap \mathbf{b}_{\text{b}}|}{|\mathbf{b}_{\text{m}} \cup \mathbf{b}_{\text{b}}|} = \frac{C + \alpha}{B + C + \beta} \geq \tau \quad (1)$$

$$\alpha \geq 0, \beta \geq 0 \quad (2)$$

We can further write the target as:

$$t := \min_{(\alpha, \beta)} \text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{b}}) = \frac{\alpha}{A + B + C + \beta} \quad (3)$$

Now it is a linear programming problem for two variables α, β. We can solve it and get the optimal solution as:

$$t^* = \max(0, \frac{\tau B + (\tau - 1) \cdot C}{A + B + C}) := \mathbb{L}_{\text{IOU}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\mathbf{m}}, \tau) \quad (4)$$

$$A = |\mathbf{b}_{\text{gt}} \setminus \mathbf{b}_{\text{m}}|, B = |\mathbf{b}_{\text{gt}} \cap \mathbf{b}_{\text{m}}|, C = |\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}}|$$

when $\beta = 0, \alpha = \max(0, \tau B + (\tau - 1) \cdot C)$. □

We use $\mathbb{L}_{\text{IOU}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\mathbf{m}}, \tau)$ to denote this new bound for IoU. Next, we present the following lemma to demonstrate that $\mathbb{L}_{\text{IOU}} > T$ is the sufficient condition of being a pruning-safe masked box.

Lemma 5. *Given a ground-truth object $\mathbf{b}_{gt}$ and the IoU pruning threshold τ, if there is one masked box $\mathbf{b}_m \in \mathcal{B}^{no}_{mask}$ satisfying that $\mathbb{L}_{\text{IOU}}(\mathbf{b}_{gt}, \mathbf{b}_m, \tau) > T$, this box is a pruning-safe masked box for object $\mathbf{b}_{gt}$ in* IOU-BOXPRUNE*, i.e., $\exists\ \mathbf{b}' \in \mathcal{B}_{robust}$ s.t. $\text{IOU}(\mathbf{b}_{gt}, \mathbf{b}') > T$.*

TABLE 5. DEFENSE PERFORMANCE OF OBJECTSEEKER WITH IOU ROBUSTNESS (ALGORITHM 3)

Dataset		PASCAL VOC		MS COCO	
Defense \ Metric	Certify class?	AP_{50}	IoU-CertR@0.8 ($T = 0.5$) far-patch	AP_{50}	IoU-CertR@0.6 ($T = 0.5$) far-patch
ObjectSeeker-YOLOR	✓	92.8%	48.3%	69.2%	37.8%
ObjectSeeker-Swin		92.0%	34.9%	68.6%	28.6%

Proof. The detected masked box $\mathbf{b}_{\text{m}} \in \mathcal{B}^{\text{no}}_{\text{mask}}$ will go through box filtering and box unionizing to generate the final output. There are three possible cases.

1) $\mathbf{b}_{\text{m}}$ is filtered in the box filtering process (Line 9). Then there is a base box $\mathbf{b}_{\text{b}} \in \mathcal{B}_{\text{base}}$ satisfying $\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}_{\text{b}}) > \tau$. From Lemma 4, we know there is a box $\mathbf{b}' = \mathbf{b}_{\text{b}} \in \mathcal{B}_{\text{base}} \subset \mathcal{B}_{\text{robust}}$ satisfies $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}') > \mathbb{L}_{\text{IOU}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{m}}, \tau) > T$.
2) $\mathbf{b}_{\text{m}}$ is removed in the box unionizing step (Line 10). This implies that there is another masked box $\mathbf{b}'_{\text{m}} \in \mathcal{B}^{\text{no-filtered}}_{\text{mask}}$ satisfying $\text{IOU}(\mathbf{b}_{\text{m}}, \mathbf{b}'_{\text{m}}) > \tau$. From Lemma 4, we know there is a box $\mathbf{b}' = \mathbf{b}'_{\text{m}} \in \mathcal{B}^{\text{no-pruned}}_{\text{mask}} \subset \mathcal{B}_{\text{robust}}$ satisfies $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}') > \mathbb{L}_{\text{IOU}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{m}}, \tau) > T$.
3) $\mathbf{b}_{\text{m}}$ is not removed and becomes a part of $\mathcal{B}^{\text{no-pruned}}_{\text{mask}} \subset \mathcal{B}_{\text{robust}}$. Then we know there is a box $\mathbf{b}' = \mathbf{b}_{\text{m}} \in \mathcal{B}_{\text{robust}}$ such that $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}') = |\mathbf{b}_{\text{gt}} \cap \mathbf{b}_{\text{m}}|/|\mathbf{b}_{\text{gt}} \cup \mathbf{b}_{\text{m}}| \geq \mathbb{L}_{\text{IOA}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{m}}, \tau) > T$. □

With Lemma 4 and Lemma 5, the certification is straightforward: we only need to replace the certification condition with the new bound $\mathbb{L}_{\text{IOU}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\mathbf{m}}, \tau) > T$ in Line 6 of Algorithm 2.

Implementation and performance evaluation. In our implementation, we consider a box is a non-overlapping box if the smallest distance between the masked box and the mask along x and y axes is larger than 5% of the range of the corresponding axis (i.e., height or width). We use the same setup as in Section 4 to evaluate Algorithm 3. We set $\tau_{\text{IoU}} = 0.8$ and the IoU certification threshold $T_{\text{IoU}} = 0.5$ and report the defense performance in Table 5. As shown in the table, Algorithm 3 has similarly high clean AP as Algorithm 1 (recall Table 2), and more importantly, achieves the first certified IoU robustness against patch hiding attacks.

Appendix C. Taxonomy of Robustness Notions against Hiding Attacks

In the main body of this paper, we focus on IoA robustness; in Appendix B, we further presented a ObjectSeeker variant that has IoU robustness against far-patch attackers. In this section, we aim to provide a taxonomy of different robustness notions against hiding attacks to shed a light on future research. We summarize four major robustness notions in Table 6, which are categorized based on two important robustness factors as discussed next.

TABLE 6. TAXONOMY OF ROBUSTNESS NOTIONS

	Attack detection	Robust prediction
IoA robustness	**Notion I** DetectorGuard [13]	**Notion II** ObjectSeeker (Section 3)
IoU robustness	**Notion III** –	**Notion IV** ObjectSeeker (Appendix B)

Factor 1: attack detection vs. robust prediction. The first factor is the defense format: attack detection versus robust prediction. An attack-detection defense aims to detect an attack: it alerts and abstains from making predictions when it detects an attack. That is, we allow the defense to output a special symbol $\perp$ for cases when it detects an attack. In contrast, a robust-prediction defense does not involve the abstention symbol $\perp$: it has to always mask robust predictions. Clearly, robust-prediction defenses achieve a stronger robustness notion – a robust-prediction defense can directly reduce to an attack-detection defense that never alerts. We note that DetectorGuard [13] is an attack-detection defense (Notion I) while ObjectSeeker aims to build a robust-prediction defense (Notion II and IV).

Factor 2: IoA robustness vs. IoU robustness. The second factor is about the guarantee for the box quality: IoA robustness versus IoU robustness. In the main body of this paper, we consider IoA robustness because it aligns with the defense objective against patch hiding attacks: we aim to detect at least part of the object, i.e., $\text{IOA}(\mathbf{b}_{\text{gt}}, \mathbf{b}') = |\mathbf{b}_{\text{gt}} \cap \mathbf{b}'|/|\mathbf{b}_{\text{gt}}| > T$.

However, a defense with IoA robustness might end up predicting large bounding boxes that cover the entire image. When this happens, the IoA robustness is satisfied (IoA equals to 1), but the defense output is not ideal: we only know that there are objects of certain classes but do not know the object locations. Therefore, we are motivated to consider the concept of IoU robustness, i.e., $\text{IOU}(\mathbf{b}_{\text{gt}}, \mathbf{b}') = |\mathbf{b}_{\text{gt}} \cap \mathbf{b}'|/|\mathbf{b}_{\text{gt}} \cup \mathbf{b}'| > T$, which adds additional constraints on the sizes of predicted boxes. In Appendix B, we discuss a variant of ObjectSeeker that can certify IoU robustness against far-patch attackers.

Remark: non-triviality of IoA robustness and high clean performance. Despite the subtle issues of IoA robustness discussed above, we note that building defense with IoA robustness and high clean performance is non-trivial. Yes, if defenders know that there is going to be an attack, they can trivially output large boxes for IoA robustness. However, in practice, we do not know if the input image is a benign normal image or an adversarially

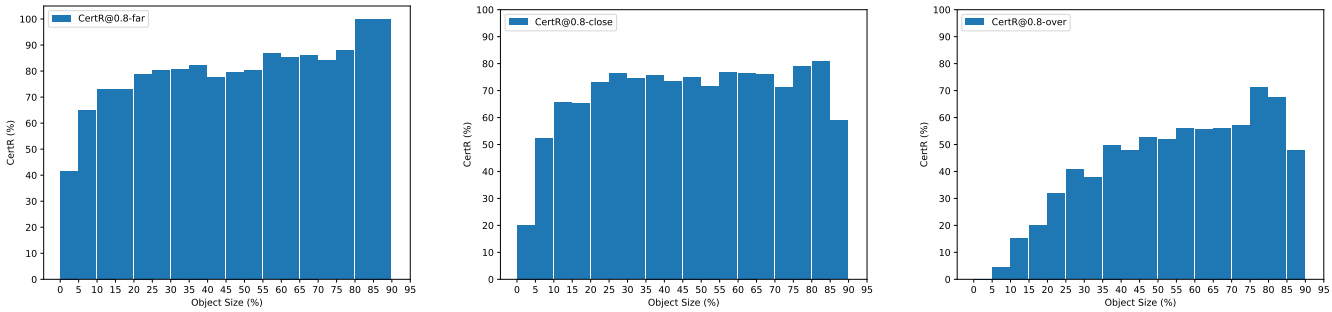

Figure 12. ObjectSeeker robustness for VOC objects of different sizes (left to right: far-patch, close-patch, over-patch)

patched image. If defenders always output large boxes, the performance on clean images will be bad: note that we use the conventional IoU to evaluate clean performance (when no attacks happen); IoA is only for robustness evaluation. Moreover, IoA robustness requires the correctness of box class labels. Achieving label correctness is also non-trivial.

Remark: the connection between Notion II and Notion I. As shown in Table 6, both Notion I and Notion II use IoA robustness, and they differ in the defense formats of attack detection versus robust prediction. Here, we can demonstrate the equivalence of two notions in terms of certified robustness.

First, we can build a Notion I defense with a Notion II defense. This is directly implied by the definition: any Notion II defense is a valid Notion I defense that relinquishes the freedom of issuing alerts.

Second, counterintuitively, we can also build a Notion II defense with a Notion I defense. The strategy is that, if the Notion I defense detects an attack, instead of issuing an alert, it outputs large bounding boxes of all different classes to cover the entire image. Since these large boxes have IoAs of 1 for any object of the same object class, the defense is considered IoA-robust.[2]

Note: empirical advantage of Notion II defenses. Despite the equivalence in certifiable robustness with Notion I defenses, Notion II defenses still have advantages in terms of empirical performance. For example, when there is a false alert in the clean setting, a Notion I based defense would either alert or output useless large boxes while a Notion II defense can still predict reasonable boxes (e.g. boxes that obtain high IoU with the ground-truth objects).

Remark: IoA and IoU robustness in ObjectSeeker. As shown in Table 6, ObjectSeeker is flexible and can be instantiated with either IoA or IoU. We focus on IoA robustness in the main body for the following reasons. First, IoA robustness is non-trivial and interesting to study (recall the first remark in this section); it intuitively aligns with the objective of mitigating hiding attacks. Second, focusing on IoA enables a fair comparison with DetectorGuard [13], whose robustness guarantee is limited to IoA robustness with $T = 0$. Third, though we have competitive IoU-CertR numbers with IoA-CertR against *far-patch* (recall Appendix B), certifiable IoU robustness against over-patch and close-patch is significantly more challenging to achieve. We will need better pruning strategies to instantiate our framework for better IoU robustness in the future.

Factor 3: Protect class labels or not. In addition to two robustness factors presented in Table 6, we can further categorize robustness notions based on whether the defense protects the class label or not. For example, DetectorGuard [13] is fundamentally limited in its design not to be able to certify class labels. In contrast, ObjectSeeker is flexible for the class label certification.

Future of robust object detection. As a summary of this section, we believe that the ultimate defense objective is a robust-prediction defense with IoU robustness (Notion IV). The defense problem can then be interpreted as a list decoding problem: we aim to output a list of bounding boxes such that a subset of bounding boxes have high IoUs with all ground-truth boxes. Furthermore, to build even stronger defenses, we can also consider a stronger attacker who also wants to increase FP errors. We can deploy an empirical defense via classifying all predicted bounding boxes and removing boxes with inconsistent labels [13].

Appendix D. Additional Discussions and Experiments

In this section, we provide quantitative discussions on objects of different sizes, absolute defense runtime, and certification threshold T.

ObjectSeeker's robustness for objects of different sizes. In this analysis, we aim to understand the relationship between robustness and object sizes. We divide VOC objects into different groups based on their sizes (occupying 0-5%, 5-10%, $\cdots$, 95-100% image pixels) and plot their CertR in Figure 12. As shown in the figure, larger objects tend to have higher robustness. This is because vanilla object detectors have a better chance to detect larger objects on masked images. We note that the CertR for over-patch is more sensitive to object sizes, partially due to that an over-patch can sometimes occlude the major part of small objects and make robust object detection hard or even impossible.

Additional discussion on absolute runtime. In Figure 9, we demonstrated that we could balance the trade-

2. We note that an extremely large box has a small IoU with the ground-truth box; therefore, this strategy of outputting large boxes does not apply to Notion III and Notion IV where we consider IoU robustness.

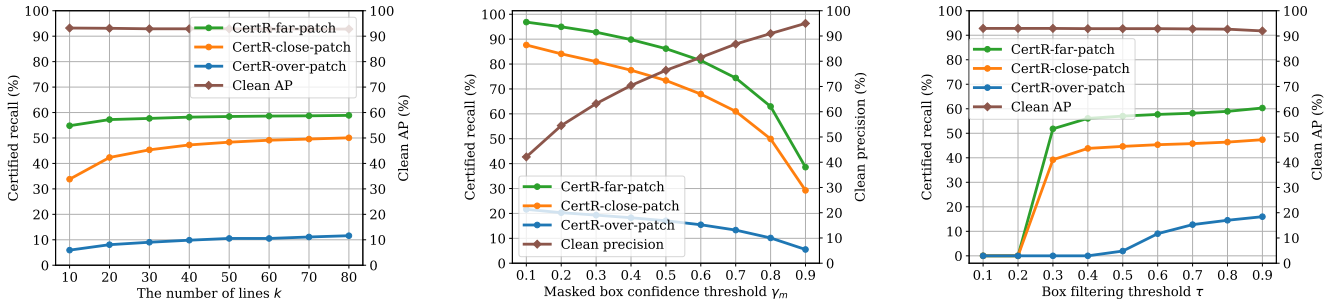

Figure 13. The effects of different hyperparameters on ObjectSeeker with larger $T = 0.2$ (VOC; left to right: k, γ_{m}, τ)

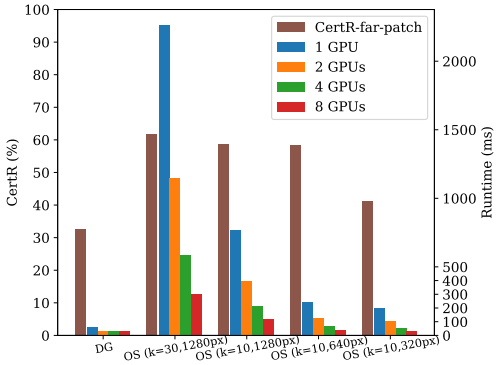

Figure 14. Wall-lock per-image runtime of DetectorGuard (DG) [13] and ObjectSeeker (OS) on VOC

off between efficiency and robustness by tuning the parameter k. In this section, we provide further discussions on implementation-level optimizations for absolute runtime. Specifically, we consider using different input image sizes and different numbers of GPUs. In Figure 14, we report runtime results for DetectorGuard [13] and ObjectSeeker using different images sizes, different numbers of NVIDIA RTX A4000 GPUs, and different k (for ObjectSeeker). First, we can see that using multiple GPUs can significantly reduce wall-clock runtime, given that the inference on masked images is trivially parallelizable. For example, if we set $k = 10$ and use an image size of 1280px, using 8 GPUs can reduce runtime from 770.6ms 117.4ms (6.6× speedup). In contrast, DetectorGuard's runtime improvement with multiple GPUs is limited (up to 2×). Second, we can see that reducing the image size can also significantly improve runtime. For example, if we resize images from 1280px (the default value used in the paper) to 640px, the runtime for ObjectSeeker with $k = 10$ on 8 GPUs improves from 117.4ms to 39.6ms (3.0× speedup), while the robustness is only slightly affected (from 58.8% to 58.4%). However, we note that further reducing the image size (to 320px) can greatly hurt the robustness (to 41.2%). We note that DetectorGuard [13]'s robustness module uses fixed image size and has limited benefit from image resizing.

In summary, Figure 14 demonstrates the feasibility to reduce absolute runtime via implementation-level optimizations. We can have a latency of 40.0ms (25fps) on VOC images using $k = 10$, an image size of 640px, and 8 GPUs, while maintaining high CertR. In practice, we should carefully configure the ObjectSeeker defense to meet computation constraints and robustness objectives, as discussed in Section 5.

Additional discussions on certification threshold T. As discussed in Section 2.3 and Section 3.3, the threshold T determines the strength of certification. In Section 4, we set the default $T = 0$ to enable a fair comparison with DetectorGuard [13]. We note that Definition 2 requires IoA $> T$ (strict inequality); thus, using $T = 0$ is non-trivial as it requires that we can at least detect any tiny part of the object. We also note that DetectorGuard is limited to $T = 0$ while ObjectSeeker is compatible with non-zero T (we reported CertR with different T in Figure 8 in Section 4.3). Here, in Figure 13, we re-analyze the effects of the three most important defense hyperparameters ($k, \gamma_{\text{m}}, \tau$) on CertR with a larger T of 0.2. As shown in the figure, the trends of curves for k and γ_{m} largely stay the same as their counterparts of $T = 0$ (Figures 5 and 6); the only difference is that the curves shift down a bit due to the stronger certification requirement. For the box filtering threshold τ, we can see that CertR drops drastically when we use smaller τ, compared to Figure 7. This is because the certification bound $\mathbb{L}_{\text{IOA}}(\mathbf{b}_{\text{gt}}, \mathbf{b}_{\text{m}}, \tau) = (|\mathbf{b}_{\text{m}}| \cdot \tau - |\mathbf{b}_{\text{m}} \setminus \mathbf{b}_{\text{gt}}|)/|\mathbf{b}_{\text{gt}}|$ becomes too low with a small τ, and thus makes it hard to certify for a non-zero T.

Remark: the choice of T in practice. The semantic meaning of T for IoA robustness is how much of the object can be detected. In practice, the choice of T for robustness *evaluation* should depend on the application. For example, if we want to perform object counting or traffic sign detection/recognition, $T = 0$ could be good enough. However, if we consider a robot safely navigating through a large number of large obstacles (without any collision), we might want to use a larger non-zero T. Moreover, it is also reasonable to consider different T at the same time. For example, we can report CertR for different T as done in Figure 8. Another option is to report the averaged CertR across different T. The reported number is approximately the area under the curve (AUC) for Figure 8 (34.5% for far-patch; 25.7% for close-patch; 4.35% for over-patch).